SCONTENTS

STORES LaTeX CONTENTS

v2.8 2026-11-01*

©2019–2026 by Pablo González L†

CTAN: https://www.ctan.org/pkg/scontents
https://github.com/pablgonz/scontents

Abstract

This package allows to store LaTeX code, including "*verbatim*", in ⟨*sequences*⟩ using the l3seq module of expl3. The ⟨*stored content*⟩ can be used as many times as desired in the document, additionally you can write to ⟨*external files*⟩ or show it in ⟨*verbatim style*⟩.

Contents

1 Description of the package 1
2 Motivation and Acknowledgments . . . 1
3 License and Requirements 2
4 The scontents package 2
4.1 Installation 2
4.2 Loading and options 2
4.3 The TAB character 2
4.4 Configuration of the options 3
4.5 Options Overview 3
5 User interface 4
5.1 The environment scontents 4
5.2 The command \newenvsc 5
5.3 The command \Scontents 5
5.4 The command \getstored 6
5.5 The command \foreachsc 6
5.6 The command \typestored 6
5.7 The command \meaningsc 7
5.8 The command \mergesc 7
5.9 The environment verbatimsc 7
6 Other commands provided 8
6.1 The command \countsc 8
6.2 The command \cleanseqsc 8
7 The scontents package in action 8
8 Examples 9
8.1 From answers package 9
8.2 From filecontentsdef package . . 9
8.3 From TeX-SX 10
8.4 Customization of verbatimsc 11
8.5 The command \mergesc in action . . 14
8.6 The tagged PDF example 15
9 Change history 16
10 Index of Documentation 18
11 References 18
12 Implementation 20
13 Index of Implementation 54

1 Description of the package

The SCONTENTS package allows to ⟨*store contents*⟩ in ⟨*sequences*⟩ or ⟨*external files*⟩. In some ways it is similar to the filecontentsdef package, with the difference in which the ⟨*content*⟩ is stored. The idea behind this package is to get an approach to ConTeXt *"buffers"* by making use of ⟨*sequences*⟩.

2 Motivation and Acknowledgments

In LaTeX there is no direct way to record content for later use, although you can do this using \macro, recording ⟨*verbatim content*⟩ is a problem, usually you can avoid this by creating external files or boxes.

The general idea of this package is to try to imitate this implementation "*buffers*" that has ConTeXt which allows you to save content in memory, including *verbatim*, to be used later. The filecontentsdef[2] package solves this problem and since expl3[1] has an excellent way to manage data, ideas were combined giving rise to this package.

This package would not be possible without the great work of Jean François Burnol who was kind enough to take my requirements into account and add the filecontentsdefmacro environment to filecontentsdef package. Also a special thanks to Phelype Oleinik who has collaborated and adapted a large part of the code and all LaTeX team for their great work and to the different members of the TeX-SX community who have provided great answers and ideas. Here a note of the main ones:

1. Stack datastructure using LaTeX

*This file describes a documentation for v2.8, last revised 2026-11-01.

†E-mail: «pablgonz@educarchile.cl».

2. LaTeX equivalent of ConTeXt buffers
3. Storing an array of strings in a command
4. Collecting contents of environment and store them for later retrieval
5. Collect contents of an environment (that contains verbatim content)

Starting with version 2.3 the SCONTENTS package is fully compatible with *tagged* PDF and will have LaTeX release 2026-06-01 (TeX Live 2026) and update `expl3` release as a minimum requirement.

3 License and Requirements

Permission is granted to copy, distribute and/or modify this software under the terms of the LaTeX Project Public License (lppl), version 1.3 or later (`https://www.latex-project.org/lppl.txt`). The software has the status "maintained".

The package SCONTENTS requires an updated version of LaTeX to work (minimum required to LaTeX release 2026-06-01 and `expl3` update). This package can be used with `plain`, `context`, `xelatex`, `lualatex`, `pdflatex` and the classical workflow `latex»dvips»ps2pdf`.

4 The `scontents` package

4.1 Installation

The package SCONTENTS is present in TeX Live and MiKTeX, use the package manager to install. For manual installation, download scontents.zip and unzip it, run `luatex scontents.ins` and move all files to appropriate locations, then run `mktexlsr`. To produce the documentation with source code run `luatex scontents.ins` and `lualatex scontents.dtx` three times.

```
scontents.tex       »  TDS:tex/generic/scontents/
scontents-code.tex  »  TDS:tex/generic/scontents/
scontents.sty       »  TDS:tex/latex/scontents/
t-scontents.mkiv    »  TDS:tex/context/third/scontents/
scontents.pdf       »  TDS:doc/latex/scontents/
README.md           »  TDS:doc/latex/scontents/
scontents.dtx       »  TDS:source/latex/scontents/
scontents.ins       »  TDS:source/latex/scontents/
```

4.2 Loading and options

The package is loaded in the usual way:

For LaTeX users

```
\usepackage[⟨key = val⟩]{scontents}
```

For plain TeX users

```
\input scontents.tex
```

For ConTeXt users

```
\usemodule{scontents}
```

The package options are not available for plain TeX and ConTeXt and must be passed using `\setupsc` (§4.4). ConTeXt users should use `-luatex`, the implementation does not support LuaMetaTeX.

4.3 The TAB character

Some users use the horizontal TAB '⇤⇥' character from keyboard to indent the source code of the document and depending on the text editor used, some will use real TAB (*hard tabs*), others with *soft tabs* (␣␣ or ␣␣␣␣) or both.

At first glance it may seem the same, but the way in which TAB character (*hard tabs*) are processed according to the context in which they are found within a file, both in ⟨*reading*⟩[1] and ⟨*writing*⟩[2] are different and may have adverse consequences.

In a standard LaTeX document, the character TAB '⇤⇥' are treated as explicit spaces (in most contexts) and is the behavior when ⟨*stored content*⟩, but when ⟨*writing files*⟩ these are preserved.

[1] Check the answer given by Ulrich Diez in Keyboard TAB character in argument v (xparse).
[2] Check the answer given by Enrico Gregorio in How to output a tabulation into a file.

With a TEX Live distribution, the TAB character is *printable* for latex, pdflatex and lualatex, but if you use xelatex you must add the -8bit option on the command line, otherwise you will get TEX character TAB '^^I' in the ⟨*output file*⟩.

As a general recommendation *"Do not use TAB character unless strictly necessary"*, for example within a *"verbatim environment"* that supports this character such as Verbatim of the packages fancyvrb[5], fvextra[7] or lstlisting of the package listings[6] or when you want to generate a MakeFile file.

4.4 Configuration of the options

Most of the options can be passed directly to the package or using the command \setupsc. All boolean keys can be passed using the equal sign '=' or just the name of the key, all unknown ⟨*keys*⟩ will return an error. In this section are described some of the options, a summary of all options is shown in §4.5.

\setupsc `\setupsc{⟨key = val⟩}`

The command \setupsc sets the ⟨*keys*⟩ in a global way, it can be used both in the preamble and in the body of the document as many times as desired. However, options set in the declaration of an environment (with \newenvsc) have precedence over options set with \setupsc.

Options in the optional arguments of environments and commands have the highest precedence, overriding both options in \newenvsc and in \setupsc.

debug-var = {⟨*true* | *false*⟩} default: *true*

Determines whether the ⟨*argument*⟩ passed to commands or the ⟨*body*⟩ passed to provided environments is written to the .log file. This ⟨*key*⟩ is only available as a package option or using \setupsc.

verb-font = {⟨*font family*⟩} default: \ttfamily

Sets the ⟨*font family*⟩ used to display the ⟨*stored content*⟩ for \typestored, \mergesc and \meaningsc commands. This ⟨*key*⟩ is only available as a package option or using \setupsc.

store-all = {⟨*seq name*⟩} default: *not used*

It is a ⟨*meta-key*⟩ that sets the store-env key of the scontents environment and the store-cmd key of the \Scontents command. This ⟨*key*⟩ is only available as a package option or using \setupsc.

overwrite = {⟨*true* | *false*⟩} default: *false*

Sets whether the ⟨*files*⟩ generated by write-out and write-env from the scontents environment will be rewritten. This ⟨*key*⟩ is available as a package option, for \setupsc, for \Scontents and \Scontents*, for the base environment scontents and all environments created using \newenvsc.

print-all = {⟨*true* | *false*⟩} default: *false*

It is a ⟨*meta-key*⟩ that sets the print-env key of the scontents environment and the print-cmd key of the \Scontents command. This ⟨*key*⟩ is only available as a package option or using \setupsc.

force-eol = {⟨*true* | *false*⟩} default: *false*

Sets if the *last end of line* for the ⟨*stored content*⟩ is hidden or not. This ⟨*key*⟩ is necessary only if the *last line* is the closing of some environment defined by fancyvrb[5] or fvextra[7] packages as \end{Verbatim} or another environment that does not support a comments '%' after closing \end{«env»}%. This ⟨*key*⟩ is available for the base environment scontents, the \Scontents and \Scontents* commands and all environments created using \newenvsc.

width-tab = {⟨*integer*⟩} default: *1*

Sets the equivalence in ⟨*spaces*⟩ for the character TAB '↹' used when displaying ⟨*stored content*⟩ in *verbatim style*. The value must be a ⟨*positive integer*⟩. This ⟨*key*⟩ is available for \typestored, \mergesc and \meaningsc commands.

4.5 Options Overview

key	package	\setupsc	scontents	\Scontents	\Scontents*	\typestored	\meaningsc	\mergesc
debug-var	✓	✓	✗	✗	✗	✗	✗	✗
verb-font	✓	✓	✗	✗	✗	✗	✗	✗
store-all	✓	✓	✗	✗	✗	✗	✗	✗
overwrite	✓	✓	✓	✓	✓	✓	✓	✓
print-all	✓	✓	✗	✗	✗	✗	✗	✗
force-eol	✓	✓	✓	✓	✓	✗	✗	✗
width-tab	✓	✓	✗	✗	✗	✓	✓	✓
store-env	✓	✓	✓	✗	✗	✗	✗	✗
store-cmd	✓	✓	✗	✓	✓	✗	✗	✗
print-env	✓	✓	✓	✗	✗	✗	✗	✗
print-cmd	✓	✓	✗	✓	✓	✓	✓	✓
write-env	✗	✗	✓	✗	✗	✗	✗	✗
write-cmd	✗	✗	✗	✓	✓	✗	✗	✗
write-out	✗	✗	✓	✓	✓	✓	✓	✓
typestored	✗	✗	✗	✗	✗	✗	✗	✓
meaningsc	✗	✗	✗	✗	✗	✗	✗	✓

©2019–2026 by Pablo González L

5 User interface

The user interface consists in `scontents` environment, `\Scontents` and `\Scontents*` commands to ⟨*stored content*⟩, `\getstored` command to get the ⟨*stored content*⟩, `\typestored` and `\mergesc` commands to print *verbatim style* the ⟨*stored content*⟩ along with other utilities described in this documentation.

5.1 The environment `scontents`

`scontents`

```
\begin{scontents}[⟨key=val⟩]
    ⟨body env⟩
\end{scontents}
```

The `scontents` environment processes {⟨*body env*⟩} and *"stores"* it in a ⟨*sequence*⟩ or *"writes"* it to an ⟨*external file*⟩ if desired, including *verbatim material.* After the package has been loaded, the environment can be used both in the preamble and in the body of the document.

For the correct operation `\begin{scontents}` and `\end{scontents}` must be in *different lines*, all ⟨*keys*⟩ must be passed separated by commas and *without spaces* '␣' of the start of the environment.

Comments '%' or *any character* after `\begin{scontents}` or [⟨*key = val*⟩] on the *same line* are NOT supported, the package will return an "error" message if this happens. In a similar way comments '%' or *any character* after `\end{scontents}` on the *same line* the package will return a "warning" message.

The environment can be *"nested"* if it is properly balanced and does not appear *"literally"* in commented lines or in some *verbatim* environment or command. As an example:

```
\begin{scontents}[store-env=outer]
This text is in the outer environment (before nested).
\begin{scontents}[store-env=inner]
This text is found in the inner environment (inside of nested).
\end{scontents}
This text is in the outer environment (after nested).
\end{scontents}
```

Of course, the ⟨*stored content*⟩ in the *sequence* {⟨*inner*⟩} is *only available* after ⟨*stored content*⟩ in the *sequence* {⟨*outer*⟩} one has been retrieved, either by using the key `print-env` or `\getstored` command.

It is advisable to store the ⟨*content*⟩ in *sequences* with different names, so as not to get lost in the order (position) in which it is stored.

Notes for plain TEX and ConTEXt users

In plain TEX there are no environments as in LATEX. Instead of using the environment `scontents`, one should use a *"pseudo environment"* delimited by `\scontents` and `\endscontents`.

`\scontents` `\endscontents`

```
\scontents[⟨key=val⟩]
    ⟨body env⟩
\endscontents
```

ConTEXt users should use `\startscontents` and `\stopscontents`.

`\startscontents` `\stopscontents`

```
\startscontents[⟨key=val⟩]
    ⟨body env⟩
\stopscontents
```

Options for environment

The environment options can be configured globally using option in package or the `\setupsc` command and locally using [⟨*key = val*⟩] in the environment. The key `force-eol` is available for this environment.

`store-env` = {⟨*seq name*⟩} default: *contents*

Sets the *name* of the *sequence* in which the {⟨*body env*⟩} will be stored. If the *sequence* does not exist, it will be created globally.

`print-env` = {⟨*true* | *false*⟩} default: *false*

Sets if the ⟨*stored content*⟩ is displayed or not at the time of running the environment. The ⟨*stored content*⟩ is extracted from the *sequence* {⟨*seq name*⟩} set by the key `store-env` at the time it is executed.

`write-env` = {⟨*file.ext*⟩} default: *not used*

Sets the *name* of the ⟨*external file*⟩ in which the {⟨*body env*⟩} of the environment will be written. The ⟨*file.ext*⟩ will be created in the working directory and the {⟨*body env*⟩} will be *stored* in the *sequence* set by the key `store-env` at the time it is executed. If ⟨*file.ext*⟩ does not exist, it will be created or overwritten if the `overwrite` key is used.

`write-out` = {⟨*file.ext*⟩} default: *not used*

©2019–2026 by Pablo González L

Sets the *name* of the ⟨*external file*⟩ in which the {⟨*body env*⟩} of the environment will be written. The ⟨*file.ext*⟩ will be created in the working directory and the {⟨*body env*⟩} will NOT be *stored* in the *sequence* set by the key `store-env` at the time it is executed. If ⟨*file.ext*⟩ does not exist, it will be created or overwritten if the `overwrite` key is used.

In the keys `write-env` and `write-out` the character `TAB` will be written in ⟨*file.ext*⟩, relative or absolute paths are not supported. XeLaTeX users using character `TAB` must add `-8bit` at the command line, otherwise you will get TeX character `TAB` '`^^I`' in ⟨*file.ext*⟩.

5.2 The command \newenvsc

`\newenvsc` `\newenvsc{⟨env name⟩}[⟨initial keys⟩]`

The command `\newenvsc` allows you to create ⟨*new environments*⟩ based on the same characteristics of the `scontents` environment. The values entered in `[⟨initial keys⟩]` will be considered as the default values for this new environment and the valid ⟨*keys*⟩ are `store-env` and `print-env`. For example:

```
\newenvsc{myenvstore}[store-env=myseq,print-env=false]
```

created the environment `myenvstore` that ⟨*stored content*⟩ in the *sequence* {⟨*myseq*⟩} and will NOT display the ⟨*stored content*⟩ when the environment it is executed.

5.3 The command \Scontents

`\Scontents` `\Scontents[⟨key = val⟩]{⟨argument⟩}`
`\Scontents*[⟨key = val⟩]{⟨argument⟩}`
`\Scontents*[⟨key = val⟩]⟨del⟩⟨argument⟩⟨del⟩`

The `\Scontents` command reads the {⟨*argument*⟩} in standard mode and *"stores"* it in the *sequence* set by the key `store-cmd` at the time it is executed. It is not possible to pass *verbatim things*, but it is possible to use the implementation of `\Verb` delimited by *braces* '`{ }`' provided by the `fvextra`[7] package for *verbatim one line* and `\lstinline` from `listings`[6] package, but it is preferable to use the *starred argument* '`*`'.

The `\Scontents*` command reads the {⟨*argument*⟩} under *verbatim catcode* regimen and *"stores"* it in the *sequence* set by the key `store-cmd` at the time it is executed. If its *"first"* delimiter is a *brace* '`{`', it will be assumed that the {⟨*argument*⟩} is nested within *braces*. Otherwise it will be assumed that {⟨*argument*⟩} is delimited by that first delimiter ⟨*del*⟩ like command `\verb`. Blank lines are preserved, escaped braces '`\{`' and '`\}`' must also be balanced if the {⟨*argument*⟩} is used with *braces* and character `TAB` typed from the keyboard are converted into spaces.

The *starred argument* '`*`' and `[⟨key = val⟩]` must NOT be separated by *spaces* '␣' between them and the command. Both versions can be used anywhere in the document and cannot be used as an {⟨*argument*⟩} for other command.

Options for command

The command options can be configured globally using option in package or the `\setupsc` command and locally using `[⟨key = val⟩]`.

`store-cmd` = {⟨*seq name*⟩} default: *contents*

Sets the *name* of the *sequence* in which the {⟨*argument*⟩} is stored. If the *sequence* does not exist, it will be created globally.

`print-cmd` = {⟨*true* | *false*⟩} default: *false*

Sets if the ⟨*stored content*⟩ is displayed or not at the time of running the command. The ⟨*stored content*⟩ is extracted from the *sequence* {⟨*seq name*⟩} set by the key `store-cmd` at the time it is executed.

Options for writing files

The following ⟨*keys*⟩ and the `force-eol` and `overwrite` keys are available for both versions of the command. The `\Scontents*` command writes the {⟨*argument*⟩} to ⟨*file.ext*⟩ exactly as it was typed. The `\Scontents` command reads the {⟨*argument*⟩} in standard mode, so it is written as a list of tokens, in the same way as `\detokenize` does: a space is added after each control word, the character '`#`' is doubled, the line breaks become spaces and the blank lines become `\par`. To write the {⟨*argument*⟩} literally, use the *starred argument* '`*`'.

`write-cmd` = {⟨*file.ext*⟩} default: *not used*

Sets the *name* of the ⟨*external file*⟩ in which the {⟨*argument*⟩} will be written. The ⟨*file.ext*⟩ will be created in the working directory and the {⟨*argument*⟩} will be *stored* in the *sequence* set by the key `store-cmd` at the time it is executed. If ⟨*file.ext*⟩ does not exist, it will be created or overwritten if the `overwrite` key is used.

©2019–2026 by Pablo González L

`write-out` = {⟨*file.ext*⟩} default: *not used*

Sets the *name* of the ⟨*external file*⟩ in which the {⟨*argument*⟩} will be written. The ⟨*file.ext*⟩ will be created in the working directory and the {⟨*argument*⟩} will NOT be *stored* in any *sequence* set by the key `store-cmd` at the time it is executed. If ⟨*file.ext*⟩ does not exist, it will be created or overwritten if the `overwrite` key is used.

In the keys `write-cmd` and `write-out` the character `TAB` will be written in ⟨*file.ext*⟩, relative or absolute paths are not supported. XeLaTeX users using character `TAB` must add `-8bit` at the command line, otherwise you will get TeX character `TAB` '`^^I`' in ⟨*file.ext*⟩.

5.4 The command `\getstored`

`\getstored` `\getstored[`⟨*index*⟩`]{`⟨*seq name*⟩`}`

The `\getstored` command retrieves the ⟨*stored content*⟩ according to the *integer value* set in ⟨*index*⟩ which corresponds to the *position* of the ⟨*stored content*⟩ in the *sequence* {⟨*seq name*⟩}.

The command is robust and can be used as an {⟨*argument*⟩} for another command. If the *optional argument* is not passed, the default value is the *"last"* ⟨*stored content*⟩ in the *sequence* {⟨*seq name*⟩}.

5.5 The command `\foreachsc`

`\foreachsc` `\foreachsc[`⟨*key = val*⟩`]{`⟨*seq name*⟩`}`

The command `\foreachsc` iterates through and executes the command `\getstored` on the ⟨*stored content*⟩ in the *sequence* {⟨*seq name*⟩}. If the *optional argument* is not passed run `\getstored` on all ⟨*stored content*⟩ in the *sequence* {⟨*seq name*⟩}.

Options for command

`sep` = {⟨*code*⟩} default: *empty*

Establishes the *separation* between each ⟨*stored content*⟩ in the *sequence* {⟨*seq name*⟩}. For example, you can use `sep={\\[10pt]}` for vertical separation of ⟨*stored contents*⟩.

`step` = {⟨*integer*⟩} default: *1*

Sets the *increment* (⟨*step*⟩) applied to the value set by key `start` for each ⟨*stored content*⟩ in the *sequence* {⟨*seq name*⟩}. The value must be a ⟨*positive integer*⟩.

`start` = {⟨*integer*⟩} default: *1*

Set the *position* of the ⟨*stored content*⟩ within the *sequence* {⟨*seq name*⟩} from which to *start* executing. The value must be a ⟨*positive integer*⟩.

`stop` = {⟨*integer*⟩} default: *total*

Set the *position* of the ⟨*stored content*⟩ within the *sequence* {⟨*seq name*⟩} at which execution ends. The value must be a ⟨*positive integer*⟩.

`before` = {⟨*code*⟩} default: *empty*

Sets the {⟨*code*⟩} that will be executed *before* each ⟨*stored content*⟩ within the *sequence* {⟨*seq name*⟩}. The {⟨*code*⟩} must be passed *between braces* '{ }'.

`after` = {⟨*code*⟩} default: *empty*

Sets the {⟨*code*⟩} that will be executed *after* each ⟨*stored content*⟩ within the *sequence* {⟨*seq name*⟩}. The {⟨*code*⟩} must be passed *between braces* '{ }'.

`wrapper` = {⟨*code* `{#1}` *more code*⟩} default: *empty*

Wraps the ⟨*stored content*⟩ within the *sequence* {⟨*seq name*⟩} referenced by `{#1}`. The {⟨*code*⟩} must be passed *between braces* '{ }'. For example `\foreachsc[wrapper={\makebox[1em][l]{#1}}]{contents}`.

5.6 The command `\typestored`

`\typestored` `\typestored[`⟨*index, start-stop, 1-end, keys*⟩`]{`⟨*seq name*⟩`}`

The command `\typestored` places the ⟨*stored content*⟩ in the *sequence* {⟨*seq name*⟩} into the internally `verbatimsc` environment (§5.9). The *integer value* set at ⟨*index*⟩ corresponds to the *position* of the ⟨*stored content*⟩ in the *sequence* {⟨*seq name*⟩} will be printed, a *negative* ⟨*index*⟩ counts from the last ⟨*stored content*⟩ as in `\getstored`, if ⟨*1-end*⟩ is used *"all"* ⟨*stored content*⟩ in the *sequence* {⟨*seq name*⟩} will be printed.

The *integer values* set by ⟨*start-stop*⟩ define the *range* of the ⟨*stored content*⟩ in the *sequence* {⟨*seq name*⟩} that will be printed, the rest of the accepted ⟨*keys*⟩ are `print-cmd` with default value `true`, `write-out`, `width-tab` and `overwrite`.

If the *optional argument* is not passed, the *first* ⟨*stored content*⟩ in the *sequence* {⟨*seq name*⟩} will be printed.

©2019–2026 by Pablo González L

In the key write-out the character TAB will be written in ⟨file.ext⟩, relative or absolute paths are not supported. XeLaTeX users using character TAB must add -8bit at the command line, otherwise you will get TeX character TAB '^^I' in ⟨file.ext⟩.

5.7 The command \meaningsc

\meaningsc

\meaningsc[⟨index, start-stop, 1-end, keys⟩]{⟨seq name⟩}

The command \meaningsc executes \meaning on the ⟨stored content⟩ in the *sequence* {⟨seq name⟩}. The ⟨index⟩, ⟨start-stop⟩, ⟨1-end⟩ and ⟨keys⟩ have the same behavior as in the command \typestored. If the *optional argument* is not passed it defaults to the first ⟨stored content⟩ in the *sequence* {⟨seq name⟩}.

5.8 The command \mergesc

\mergesc

\mergesc[⟨typestored | meaningsc, keys⟩]{{⟨seq A⟩}[⟨index⟩], {⟨seq B⟩}[⟨start - stop⟩], {⟨seq C⟩}[⟨1-end⟩]}

The command \mergesc assembles the ⟨stored content⟩ in the *sequences* {⟨seq A⟩}[1], {⟨seq B⟩}[2-5] and {⟨seq C⟩}[1-end] and then executes \typestored (§5.6) if the typestored key is active or \meaningsc (§5.7) if the meaningsc key is active.

The {⟨argument⟩} taken by this command is a *comma separated list* of the form {⟨seq name⟩} followed by either [⟨index⟩], [⟨start-stop⟩] or [⟨1-end⟩], a {⟨seq name⟩} alone assembles all its ⟨stored content⟩. The use of the keys typestored or meaningsc are *"mandatory"* and disjoint from each other, the rest of the accepted ⟨keys⟩ are print-cmd, write-out, width-tab and overwrite.

The use of the write-out key with this command follows the same rules already described, the main advantage is that allows joining ⟨stored content⟩ *without rewriting* the file over and over again, by design TeX does not have an *append mode* for writing files, this effectively allows you to write chunks of code and then merge them into a single file.

5.9 The environment verbatimsc

verbatimsc

The environment used by \typestored and \mergesc to display the ⟨stored content⟩ in *verbatim style*. The environment is compatible with *tagged* PDF and can be customized in the following ways after loading the SCONTENTS package:

Using the packages fvextra[7] or fancyvrb[5]:

```
\ExplSyntaxOn
\cs_undefine:N \verbatimsc
\cs_undefine:N \endverbatimsc
\ExplSyntaxOff
\usepackage{fancyvrb}
\DefineVerbatimEnvironment{verbatimsc}{Verbatim}{numbers=left}
```

Using the package minted[8]:

```
\ExplSyntaxOn
\cs_undefine:N \verbatimsc
\cs_undefine:N \endverbatimsc
\ExplSyntaxOff
\usepackage{minted}
\newminted{tex}{linenos}
\newenvironment{verbatimsc}{\VerbatimEnvironment\begin{texcode}}{\end{texcode}}
```

Using the package listings[6]:

```
\ExplSyntaxOn
\cs_undefine:N \verbatimsc
\cs_undefine:N \endverbatimsc
\ExplSyntaxOff
\usepackage{listings}
\lstnewenvironment{verbatimsc}
  {
   \lstset{
           basicstyle=\small\ttfamily,
           columns=fullflexible,
           language=[LaTeX]TeX,
           numbers=left,
           numberstyle=\tiny\color{gray},
           keywordstyle=\color{red}
          }
  }{}
```

©2019–2026 by Pablo González L

At the moment, the fvextra[7] and fancyvrb[5] packages partially support *tagged* PDF.

6 Other commands provided

6.1 The command \countsc

\countsc `\countsc{⟨seq name⟩}`

The command \countsc counts a number of ⟨*stored content*⟩ in the *sequence* {⟨*seq name*⟩}.

6.2 The command \cleanseqsc

\cleanseqsc `\cleanseqsc{⟨seq name⟩}`

The command \cleanseqsc removes all ⟨*stored content*⟩ in the *sequence* {⟨*seq name*⟩}.

7 The SCONTENTS package in action

Remember the abstract on the first page? This is it:

Abstract

This package allows to store LaTeX code, including "*verbatim*", in ⟨*sequences*⟩ using the l3seq module of expl3. The ⟨*stored content*⟩ can be used as many times as desired in the document, additionally you can write to ⟨*external files*⟩ or show it in ⟨*verbatim style*⟩.

And the description of the package?

The SCONTENTS package allows to ⟨*store contents*⟩ in ⟨*sequences*⟩ or ⟨*external files*⟩. In some ways it is similar to the filecontentsdef package, with the difference in which the ⟨*content*⟩ is stored. The idea behind this package is to get an approach to ConTeXt *"buffers"* by making use of ⟨*sequences*⟩.

I've only written:

```
\begin{abstract}
This package allows to store \hologo{LaTeX} code, including \enquote{\emph{verbatim}},
in \mymeta{sequences} using the \mypkg{l3seq} module of \mypkg{expl3}. The \mymeta{stored
content} can be used as many times as desired in the document, additionally you can write
to \mymeta{external files} or show it in \mymeta{verbatim style}.
\end{abstract}
```

and

```
The \mypkg*[no-index]{scontents} package allows to \mymeta{store contents} in \mymeta{sequences}
or \mymeta{external files}. In some ways it is similar to the \mypkg{filecontentsdef}
package, with the difference in which the \mymeta{content} is stored. The idea behind
this package is to get an approach to \hologo{ConTeXt} \emph{\enquote{buffers}} by
making use of \mymeta{sequences}.
```

Of course, I didn't copy and paste. The real code they were written with is:

```
\begin{scontents}[store-env=abstract,print-env=true]
\begin{abstract}
This package allows to store \hologo{LaTeX} code, including \enquote{\emph{verbatim}},
in \mymeta{sequences} using the \mypkg{l3seq} module of \mypkg{expl3}. The \mymeta{stored
content} can be used as many times as desired in the document, additionally you can write
to \mymeta{external files} or show it in \mymeta{verbatim style}.
\end{abstract}
\end{scontents}
```

and

```
\begin{scontents}[store-env=description, print-env=true]
The \mypkg*[no-index]{scontents} package allows to \mymeta{store contents} in \mymeta{sequences}
or \mymeta{external files}. In some ways it is similar to the \mypkg{filecontentsdef}
package, with the difference in which the \mymeta{content} is stored. The idea behind
this package is to get an approach to \hologo{ConTeXt} \emph{\enquote{buffers}} by
making use of \mymeta{sequences}.
\end{scontents}
```

I stored the content in memory and then ran \getstored and \typestored. This is one of the ways you can use SCONTENTS.

©2019–2026 by Pablo González L

8 Examples

These are some adapted examples that have served as inspiration for the creation of this package. The examples are attached to this documentation and can be extracted from your PDF viewer or from the command line by running:

```
$ pdfdetach -saveall scontents.pdf
```

and then you can use the excellent arara[3] tool to compile them.

8.1 From answers package

Example 1

Adaptation of example 1 of the package answers[17]. Source file .

```
% arara: lualatex: { branch: developer }
% arara: clean: { extensions: [ aux, log ] }
\documentclass{article}
\usepackage[store-cmd=solutions]{scontents}
\newtheorem{ex}{Exercise}
\setlength{\parindent}{0pt}
\pagestyle{empty}
\begin{document}
\section{Problems}
\begin{ex}
First exercise
\Scontents{First solution.}
\end{ex}

\begin{ex}
Second exercise
\Scontents{Second solution.}
\end{ex}

\section{Solutions}
\foreachsc[sep={\\[10pt]}]{solutions}
\end{document}
```

8.2 From filecontentsdef package

Example 2

Adaptation of example from package filecontentsdef[2]. Source file .

```
% arara: lualatex: { branch: developer }
% arara: clean: { extensions: [ aux, log ] }
\documentclass{article}
\usepackage[store-env=defexercise,store-cmd=defexercise]{scontents}
\setlength{\parindent}{0pt}
\pagestyle{empty}
\begin{document}
% not starred
\Scontents{
Prove that \[x^n+y^n=z^n\] is not solvable in positive integers if $n$ is at
most $-3$.\par
}
% starred
\Scontents*|Refute the existence of black holes in less than $140$ characters.|
% write environment to \jobname.txt
\begin{scontents}[write-env=\jobname.txt]
\def\NSA{NSA}%
Prove that factorization is easily done via probabilistic algorithms and
advance evidence from knowledge of the names of its employees in the
seventies that the \NSA\ has known that for 40 years.\par
\end{scontents}
% see all stored
\begin{itemize}
\foreachsc[before={\item }]{defexercise}
\end{itemize}
```

[3] The cool TEX automation tool: https://www.ctan.org/pkg/arara

```
% \getstored are robust :)
\section{\getstored[2]{defexercise}}
\end{document}
```

8.3 From TeX-SX

Example 3

Adapted from LaTeX equivalent of ConTeXt buffers. Source file.

```
% arara: lualatex: { branch: developer }
% arara: clean: { extensions: [ aux, log ] }
\documentclass{article}
\usepackage[store-cmd=tikz]{scontents}
\usepackage{tikz}
\setlength{\parindent}{0pt}
\pagestyle{empty}
\Scontents{\matrix{ \node (a) {$a$} ; & \node (b) {$b$} ; \\ } ;}
\Scontents{\matrix[ampersand replacement=\&]
{ \node (a) {$a$} ; \& \node (b) {$b$} ; \\ } ;}
\Scontents{\matrix{\node (a) {$a$} ; & \node (b) {$b$} ; \\ } ; }
\begin{document}
\section{tikzpicture}
\begin{tikzpicture}
\getstored[1]{tikz}
\end{tikzpicture}

\begin{tikzpicture}
\getstored[2]{tikz}
\end{tikzpicture}

\begin{tikzpicture}
\getstored{tikz}
\end{tikzpicture}

\begin{scontents}[store-env=buffer]
Hello World!

This is a \verb*|fake poor man's buffer :)|.
\end{scontents}

\section{source tikz}
\typestored[1]{tikz}
\typestored[2]{tikz}
\typestored[3]{tikz}

\section{fake buffer}
\subsection{real content}
\getstored[1]{buffer}
\subsection{verbatim style}
\typestored[1]{buffer}
\subsection{meaning}
\meaningsc[1]{buffer}

\section{tikz again}
\foreachsc[before={\begin{tikzpicture}},after={\end{tikzpicture}},sep={\\[10pt]}]{tikz}
\end{document}
```

Example 4

Adapted from Collecting contents of environment and store them for later retrieval. Source file.

```
% arara: lualatex: { branch: developer }
% arara: clean: { extensions: [ aux, log ] }
\documentclass{article}
\usepackage{scontents}
\setlength{\parindent}{0pt}
\pagestyle{empty}
\begin{document}
\begin{scontents}[store-env=main]
Something for main A.
\end{scontents}
```

```
\begin{scontents}[store-env=main]
Something for \verb|main B|.
\end{scontents}

\begin{scontents}[store-env=other]
Something for \verb|other|.
\end{scontents}

\textbf{Let's print them}

This is first stored in main: \getstored[1]{main}\par
This is second stored in main: \getstored{main}\par
This is stored in other: \getstored{other}

\textbf{Print all of stored in main}\par
\foreachsc[sep={\\[10pt]}]{main}
\end{document}
```

Example 5

Adapted from Collect contents of an environment (that contains verbatim content). Source file.

```
% arara: lualatex: { branch: developer }
% arara: clean: { extensions: [ aux, log ] }
\documentclass{article}
\usepackage{scontents}
\setlength{\parindent}{0pt}
\pagestyle{empty}
\begin{document}
\section{Problem stated the first time}
\begin{scontents}[print-env=true,store-env=problem]
This is normal text.
\verb|This is from the verb command.|
\verb*|This is from the verb* command.|
This is normal text.
\begin{verbatim}
This is from the verbatim environment:
&%{}~
\end{verbatim}
\end{scontents}
\section{Problem restated}
\getstored[1]{problem}
\section{Problem restated once more}
\getstored[1]{problem}
\end{document}
```

Example 6

Adapted from Environment hiding its content. Source file.

```
% arara: lualatex: { branch: developer }
% arara: clean: { extensions: [ aux, log ] }
\documentclass[10pt]{article}
\usepackage{scontents}
\newenvsc{forshort}[store-env=forshort,print-env=false]
\setlength{\parindent}{0pt}
\pagestyle{empty}
\begin{document}

Something in the whole course.

\begin{forshort}
    Just a summary...
\end{forshort}

\end{document}
```

8.4 Customization of `verbatimsc`

Example 7

Customization of `verbatimsc` using the `fvextra`[7] and `tcolorbox`[14] packages. Source file.

```
% arara: lualatex: { branch: developer }
% arara: clean: { extensions: [ aux, log ] }
\documentclass{article}
\usepackage{scontents}
\ExplSyntaxOn
\cs_undefine:N \verbatimsc
\cs_undefine:N \endverbatimsc
\ExplSyntaxOff
\usepackage{fvextra}
\usepackage{xcolor}
\definecolor{mygray}{gray}{0.9}
\usepackage{tcolorbox}
\newenvironment{verbatimsc}%
{\VerbatimEnvironment
\begin{tcolorbox}[colback=mygray, boxsep=0pt, arc=0pt, boxrule=0pt]
\begin{Verbatim}[fontsize=\scriptsize, breaklines, breakafter=*, breaksymbolsep=0.5em,
breakaftersymbolpre={\,\tiny\ensuremath{\rfloor}}]}%
{\end{Verbatim}%
\end{tcolorbox}}
\setlength{\parindent}{0pt}
\pagestyle{empty}
\begin{document}
\section{Test \texttt{\textbackslash begin\{scontents\}} with \texttt{fancyvrb}}
Test \verb+{scontents}+ \par

\begin{scontents}
Using \verb+scontents+ env no \verb+[key=val]+, save in seq \verb+contents+
with index 1.

Prove new \Verb*{ fancyvrb with braces } and environment \verb+Verbatim*+
\begin{verbatim}
 verbatim  environment
\end{verbatim}
\end{scontents}

\section{Test \texttt{\textbackslash Scontents} with \texttt{fancyvrb}}
\Scontents{ We have coded this in \LaTeX: $E=mc^2$.}

\section{Test \texttt{\textbackslash getstored}}
\getstored[1]{contents}\par
\getstored{contents}

\section{Test \texttt{\textbackslash meaningsc}}
\meaningsc[1]{contents}\par
\meaningsc[2]{contents}

\section{Test \texttt{\textbackslash typestored}}
\typestored[1]{contents}
\typestored[2]{contents}
\end{document}
```

Example 8

Customization of `verbatimsc` using the `listings`[6] package. Source file.

```
% arara: lualatex: { branch: developer }
% arara: clean: { extensions: [ aux, log ] }
\documentclass{article}
\usepackage{scontents}
\ExplSyntaxOn
\cs_undefine:N \verbatimsc
\cs_undefine:N \endverbatimsc
\ExplSyntaxOff
\usepackage{xcolor}
\usepackage{listings}
\lstnewenvironment{verbatimsc}
  {
   \lstset{
           basicstyle=\small\ttfamily,
           breaklines=true,
           columns=fullflexible,
           language=[LaTeX]TeX,
```

```
            numbers=left,
            numbersep=1em,
            numberstyle=\tiny\color{gray},
            keywordstyle=\color{red}
           }
  }{}
\setlength{\parindent}{0pt}
\pagestyle{empty}
\begin{document}
\section{Test \texttt{\textbackslash begin\{scontents\}} with \texttt{listings}}
Test \verb+{scontents}+ \par

\begin{scontents}
Using \verb+scontents+ env no \verb+[key=val]+, save in seq \verb+contents+ with index 1.\par

Prove \lstinline[basicstyle=\ttfamily]| lstinline | and environment \verb+Verbatim*+
\begin{verbatim}
  verbatim  environment
\end{verbatim}
\end{scontents}

\section{Test \texttt{\textbackslash Scontents*} with \texttt{listings}}

\Scontents*+We have coded this in \lstinline[basicstyle=\ttfamily]|\LaTeX: $E=mc^2$|
and more.+

\section{Test \texttt{\textbackslash getstored}}
\getstored{contents}\par
\getstored[1]{contents}

\section{Test \texttt{\textbackslash typestored}}
\typestored[1]{contents}
\typestored[2]{contents}
\end{document}
```

Example 9

Customization of verbatimsc using the minted[8] package. Source file.

```
% arara: lualatex: { branch: developer }
% arara: clean: { extensions: [ aux, log ] }
\documentclass{article}
\usepackage{scontents}
\ExplSyntaxOn
\cs_undefine:N \verbatimsc
\cs_undefine:N \endverbatimsc
\ExplSyntaxOff
\usepackage{minted}
\newminted{tex}{linenos}
\newenvironment{verbatimsc}{\VerbatimEnvironment\begin{texcode}}{\end{texcode}}
\pagestyle{empty}
\setlength{\parindent}{0pt}
\begin{document}
\section{Test \texttt{\textbackslash begin\{scontents\}} with \texttt{minted}}
Test \verb+{scontents}+ \par

\begin{scontents}[overwrite,write-env=\jobname.tsc,force-eol=true]
Using \verb+scontents+ env no \verb+[key=val]+, save in seq \verb+contents+
with index 1.\par

Prove new \Verb*{ new fvextra with braces } and environment \verb+Verbatim*+
\begin{Verbatim}[obeytabs, showtabs, tab=\rightarrowfill, tabcolor=red]
No tab
        One real tab
                Two real Tab plus       one tab
\end{Verbatim}
\end{scontents}

\section{See \Verb{\jobname.tsc}}
Read \Verb{\jobname.tsc} (shows TABs as red arrows):
\VerbatimInput[obeytabs, showtabs, tab=\rightarrowfill, tabcolor=red]{\jobname.tsc}

```

©2019–2026 by Pablo González L

```
\section{Test \texttt{\textbackslash Scontents} with \texttt{minted}}

\Scontents{ We have coded \par this in \LaTeX: $E=mc^2$.}

\section{Test \texttt{\textbackslash getstored}}
\getstored[1]{contents}\par
\getstored{contents}

\section{Test \texttt{\textbackslash typestored}}
\typestored[1]{contents}
\typestored[2]{contents}
\end{document}
```

8.5 The command \mergesc in action

The command \mergesc in action, adapted from Denis Bitouzé request at https://github.com/pablgonz/scontents/issues/2. Source file .

```
% arara: lualatex: { branch: developer }
% arara: clean: { extensions: [ aux, log ] }
\documentclass{article}
\usepackage{scontents}
% Fix part of a MCE that should go before babel's loading
\begin{scontents}[store-env=mce]
\documentclass[french]{article}
\usepackage[T1]{fontenc}
\usepackage[utf8]{inputenc}
\usepackage{lmodern}
\usepackage[a4paper]{geometry}
\end{scontents}
% Fix part of a MCE that should go after (>=) babel's loading
\begin{scontents}[store-env=mce]
\usepackage{babel}
\begin{document}
\end{scontents}
% Fix part of a MCE that should go after its body
\begin{scontents}[store-env=mce]
\end{document}
\end{scontents}
\begin{document}
\section{First answer}
% Variable part of a MCE that should added to the fixed preamble, before babel's loading
\begin{scontents}[store-env=mce-1]
\usepackage{amsmath}
\end{scontents}
% Variable part of a MCE being the code snippet
\begin{scontents}[store-env=mce-1]
\begin{align}
  0 & \neq 1 \\
  1 & \neq 0
\end{align}
\end{scontents}
\begin{description}
\item[Preamble's addition]\leavevmode
  \typestored[1]{mce-1}
\item[Code snippet]\leavevmode
  \typestored[2]{mce-1}
\item[MCE]\leavevmode
  \mergesc[typestored, print-cmd=true]
    {
      {mce}[1], {mce-1}[1], {mce}[2], {mce-1}[2], {mce}[3]
    }
\end{description}
\section{Second answer}
% Variable part of a MCE that should added to the fixed preamble, before babel's loading
\begin{scontents}[store-env=mce-2]
\usepackage{amsmath}
\end{scontents}
% Variable part of a MCE being the code snippet
\begin{scontents}[store-env=mce-2]
\begin{flalign}
  0 & \neq 1 \\
```

```
  1 & \neq 0
\end{flalign}
\end{scontents}

\begin{description}
\item[Preamble's addition]\leavevmode
  \typestored[1]{mce-2}
\item[Code snippet]\leavevmode
  \typestored[2]{mce-2}
\item[MCE]\leavevmode
  \mergesc[typestored, print-cmd=true, write-out=mce.txt, overwrite=true]
    {
      {mce}[1], {mce-2}[1], {mce}[2], {mce-2}[2], {mce}[3]
    }
\end{description}
\end{document}
```

8.6 The tagged PDF example

This example is just to show the compatibility of SCONTENTS with *tagged* PDF using `lualatex`. The attached files are only for testing. Source file and *tagged* PDF.

```
% arara: lualatex: { branch: developer }
% arara: clean: { extensions: [ aux, log ] }
\DocumentMetadata{tagging=on, lang=en-US, pdfversion=2.0, pdfstandard=ua-2}
\documentclass{article}
\usepackage{scontents,unicode-math,hyperref}
\hypersetup{pdftitle = {Test scontents package},}
\begin{document}
Some

\begin{scontents}[print-env=true]
    First code \verb|\foo|

    And more code \verb|\bar|
\end{scontents}

Text

\begin{scontents}[print-env=true]
    Second code \verb|\foo|

    And more code \verb|\bar|
\end{scontents}

Text

\Scontents*{code \verb|\baz|}

% \typestored
\typestored[1]{contents}

% \mergesc
\mergesc[typestored]{ {contents}[1-end] }

% \getstored
\getstored[2]{contents}
\end{document}
```

This example has been checked using veraPDF together with ngpdf.

9 Change history

In this section you will find some (not all) of the changes in SCONTENTS development, from the first public implementation using the `filecontentsdef`[2] package to the current version with only `expl3`[1].

v2.8 (ctan), 2026-11-01
- Fix warning for empty argument in `\Scontents` and `\Scontents*`.
- Add error for empty sequence names and undefined sequences.
- Check positive integers in `start`, `step`, `stop` and `width-tab` keys.
- The messages for writing and overwriting files only go to the `.log` file.
- Fix `wrapper` key and allow `\par` in the optional argument of `\foreachsc`.
- Fix negative ⟨*index*⟩ in `\typestored`, `\meaningsc` and `\mergesc`.
- The "error" `index-out-of-range` shows the limit of the sequence and the "errors" `index-out-of-range` and `empty-variable` have a help text.
- Fix {⟨*seq name*⟩} without ⟨*index*⟩, undefined sequences and ⟨*index*⟩ out of range in `\mergesc`.
- Fix TAB at the beginning of a line in `\typestored` and `\mergesc`.

v2.7 (ctan), 2026-06-01
- Internal environment `verbatimsc` update for *tagged* PDF.
- Update requirement to LaTeX release 2026-06-01.
- Add `debug-var` key for writing variables to the `.log` file.
- Update and improvements documentation and examples.

v2.6 (ctan), 2025-11-20
- Internal environment `verbatimsc` update for *tagged* PDF.
- Update requirement to LaTeX release 2025-11-01.

v2.5 (ctan), 2025-11-01
- Replacing `\scantokens` (primitive) with `\tl_retokenize:n`.
- Cleanup warnings and details returned by `expltools`.
- Update minimum required of `expl3` to 2025-07-08.

v2.4 (ctan), 2025-05-15
- Optimization of expansion code from 'x' to 'e'.
- Restructuring code for documentation and implementation.
- Add new keys for `\typestored` and `\meaningsc`.
- Check the version of `expl3` in plain TeX and ConTeXt.

v2.3 (ctan), 2025-04-23
- Adapting the `verbatimsc` environment for *tagged* PDF.
- Update minimum required to LaTeX release 2024-11-01.
- Safer code for replacement `\obeyedline`.

v2.2 (ctan), 2025-03-26
- Fix internal definition for some functions.
- Replace `\peek_charcode_ignore_spaces:NTF` by `\peek_charcode:NTF`.
- Set correct code for `\obeyedline` implement in LaTeX release 2024-06-01.

v2.1 (ctan), 2024-06-14
- Fix `\cleanseqsc` command.
- Add `\mergesc` command.
- Fix internal definition for seq var.
- Fix internal code for `\typestored`.
- Replace `\cs_argument_spec:N` by `\cs_parameter_spec:N`.
- Detect `l3keys2e` package (obsolete in june 2022 LaTeX release).
- Minor adjustments in the documentation.

v2.0 (ctan), 2022-04-04
- Adapting the `verbatimsc` environment (compatibility `verbatim` package).
- Removed compatibility layer for older LaTeX releases.
- Fix loader in plain TeX and ConTeXt.
- Minor adjustments in the documentation.

v1.9 (ctan), 2020-01-21
- Update and improvements in the internal code.
- Updating the generic code for `I/O` verification.
- Add `write-cmd` and `write-out` keys for `\Scontents*`.
- Fix `sep` key in `\foreachsc`.

v1.8 (ctan), 2019-11-18
- Add `\newenvsc` command.
- Fix nested environment in plain TeX and ConTeXt.
- Modified default value in `\getstored`.
- Add `overwrite` key to reduce `I/O` operations.
- Deleted an unnecessary group in the code.

v1.7 (ctan), 2019-10-29
- The `verbatimsc` environment was rewritten.
- Minor adjustments in documentation.

v1.6 (ctan), 2019-10-26
- The internal behavior of `\getstored` has been modified.
- The internal behavior of `\foreachsc` has been modified.
- Corrected file extension for ConTeXt.
- Remove spurious warning.

v1.5 (ctan), 2019-10-24
- Add support for plain TeX and ConTeXt.
- Split internal code for optimization.
- Add support for vertical spaces in [⟨*key* = *val*⟩].
- Add `\foreachsc` command.
- Check if `verbatim` package is loaded.

v1.4 (ctan), 2019-10-03
- Add `store-all` key.
- Messages and keys were separated.
- Restructuring of documentation.
- Now the version of `expl3` is checked instead of `xparse`.
- The internal behavior of `force-eol` has been modified.

v1.3 (ctan), 2019-09-24
- The environment `scontents` can now nest.
- Added `force-eol`, `verb-font` and `width-tab` keys.
- The extra space has been removed when you run `\getstored`.
- Internal code has been rewritten more efficiently.
- Remove *starred argument* '`*`' for `\typestored`.
- Remove `filecontentsdef` dependency.
- Changing `\regex_replace_all:` for `\tl_replace_all:`.

v1.2 (ctan), 2019-08-28
- Restructuring of documentation.
- Added copy of `\tex_scantokens:`.

v1.1 (ctan), 2019-08-12
- Extension of documentation.
- Replace `\tex_endinput:D` by `\file_input_stop:`.

v1.0 (ctan), 2019-07-30
- First public release.

©2019–2026 by Pablo González L

10 Index of Documentation

The italic numbers denote the pages where the corresponding entry is described.

C

Commands provided by SCONTENTS:
- \Scontents 5
- \cleanseqsc 8
- \countsc 8
- \foreachsc 6
- \getstored 6
- \meaningsc 7
- \mergesc 7
- \newenvsc 5
- \setupsc 3
- \typestored 6

E

Environments :
- filecontentsdefmacro 1
- lstlisting 3
- Verbatim 3

Environments provided by SCONTENTS:
- scontents 4

K

Keys for \foreachsc provided by SCONTENTS:
- after 6
- before 6
- sep 6
- start 6
- step 6
- stop 6
- wrapper 6

Keys for \Scontents provided by SCONTENTS:
- print-cmd 5
- store-cmd 5
- write-cmd 5
- write-out 6

Keys for \setupsc provided by SCONTENTS:
- debug-var 3
- force-eol 3
- overwrite 3
- print-all 3
- store-all 3
- verb-font 3
- width-tab 3

Keys for environments provided by SCONTENTS:
- print-env 4
- store-env 4
- write-env 4
- write-out 4

P

Packages :
- answers 9
- expl3 . . . . . 1, 2, 8, 16, 17
- fancyvrb 3, 7, 8
- filecontentsdef 1, 8, 9, 16, 17
- fvextra 3, 5, 7, 8, 11
- l3keys2e 16
- l3seq 1, 8
- listings 3, 5, 7, 12
- minted 7, 13
- tcolorbox 11
- verbatim 16
- xparse 17

11 References

[1] The LaTeX Project. “The expl3 package”. Available from CTAN, https://www.ctan.org/pkg/expl3, 2026.

[2] Burnol, Jean François. “The filecontentsdef package”. Available from CTAN, https://ctan.org/pkg/filecontentsdef, 2019.

[3] The LaTeX Project. “The xparse package”. Available from CTAN, https://www.ctan.org/pkg/xparse, 2024.

[4] Niederberger, Clemens. “xsim – eXercise Sheets IMproved”. Available from CTAN, https://www.ctan.org/pkg/xsim, 2023.

[5] Van Zandt, Timothy. “The fancyvrb package - Fancy Verbatims in LaTeX”. Available from CTAN, https://www.ctan.org/pkg/fancyvrb, 2026.

[6] Hoffmann, Jobst. “The listings package”. Available from CTAN, https://www.ctan.org/pkg/listings, 2025.

[7] Poore, Geoffrey M. “The fvextra package - Extensions and patches for fancyvrb”. Available from CTAN, https://www.ctan.org/pkg/fvextra, 2026.

[8] Poore, Geoffrey M. “The minted package - Highlighted source code in LaTeX”. Available from CTAN, https://www.ctan.org/pkg/minted, 2026.

[9] The LaTeX Project. “The LaTeX3 Interfaces”. Available from CTAN, https://www.ctan.org/pkg/l3kernel, 2026.

[10] The LaTeX Project. “The LaTeX 2_ε sources”. Available from CTAN, https://ctan.org/tex-archive/macros/latex/base, 2026.

©2019–2026 by Pablo González L

[11] The LaTeX Project. "LaTeX for authors current version". Available from CTAN, https://ctan.org/pkg/latex-base, 2026.

[12] Fischer, Ulrike. "tagpdf – LaTeX kernel code for PDF tagging". Available from CTAN, https://www.ctan.org/pkg/tagpdf, 2026.

[13] The LaTeX Project. "latex-lab – LaTeX laboratory". Available from CTAN, https://www.ctan.org/pkg/latex-lab, 2026.

[14] F. Sturm, Thomas. "tcolorbox – Coloured boxes, for LaTeX examples and theorems, etc". Available from CTAN, https://ctan.org/pkg/tcolorbox, 2026.

[15] The LaTeX Project. "verbatim – Reimplementation of and extensions to LaTeX verbatim". Available from CTAN, https://www.ctan.org/pkg/verbatim, 2023.

[16] Wright, Joseph. "Programming key–value in expl3". Available from TUGboat, https://www.tug.org/TUGboat/tb31-1/tb97wright-l3keys.pdf, 2010.

[17] Wright, Joseph. "answers – Setting questions (or exercises) and answers". Available from CTAN, https://ctan.org/pkg/answers, 2014.

©2019–2026 by Pablo González L

12 Implementation

The most recent publicly released version of SCONTENTS is available at CTAN: https://www.ctan.org/pkg/scontents. Historical and developmental versions are available at https://github.com/pablgonz/scontents. While general feedback via email is welcomed, specific bugs or feature requests should be reported through the issue tracker: https://github.com/pablgonz/scontents/issues.

All variables and functions defined in this package are private and are NOT intended to work or be used by another package or module.

12.1 General conventions

The package is split into a *loader* and a *core*. The loader is different for each format and generates scontents.sty for LaTeX, scontents.tex for plain TeX and t-scontents.mkiv for ConTeXt, the core is shared by the three formats and generates scontents-code.tex, which is loaded by each loader.

Since the core runs in plain TeX and ConTeXt, where the names of some primitives may be redefined by the format, the core uses the \tex_...:D names for the primitives that have no expl3 interface, such as \tex_kern:D, \tex_lastskip:D or \tex_spacefactor:D.

Each command and environment is implemented in the same order: its variables, its messages, its ⟨*keys*⟩, its internal functions and finally the public command or environment.

12.2 Declaration of the package

First we set up the module name for DocStrip l3doc class:

```
⟨@@=scontents⟩
```

Now we define some common macros to hold the package date and version:

```
⟨*loader⟩
\def\ScontentsFileDate{2026-11-01}%
\def\ScontentsFileVersion{2.8}%
\def\ScontentsFileDescription{Stores LaTeX contents in memory or files}%
```

The LaTeX loader is quite simple, we just need to make sure of the minimum version for correct operation and then set interfaces up. The choice of LaTeX release 2026-06-01 is the latest available in TeX Live 2026 and is necessary to be able to implement the package's full compatibility with *tagged* PDF.

```
⟨*latex⟩
\NeedsTeXFormat{LaTeX2e}[2026-06-01]
\ProvidesExplPackage
  {scontents} {\ScontentsFileDate} {\ScontentsFileVersion}
  {\ScontentsFileDescription}
⟨/latex⟩
```

12.2.1 Load expl3-generic and xparse-generic in plain TeX and ConTeXt

\c__scontents_version_str
\l__scontents_char_value_int

The plain TeX and ConTeXt loaders are similar. We define the constant string \c__scontents_version_str with the version info and add \ExplSyntaxOn to be able to load the frozen xparse[3].

```
⟨*!latex⟩
⟨context⟩\writestatus{loading}{User Module scontents v\ScontentsFileVersion}
⟨context⟩\unprotect
\input expl3-generic.tex
\ExplSyntaxOn
\str_const:ce { c__scontents_version_str } { \ScontentsFileDate\space
  v\ScontentsFileVersion\space \ScontentsFileDescription }
\iow_log:e { Package: ~ scontents ~ \str_use:c { c__scontents_version_str } }
```

Outside of LaTeX we can't use ltcmd[10], part of the LaTeX kernel, but the old xparse[3] provides xparse-generic.tex, which is loadable in any format. The integer variable \l__scontents_char_value_int saves the category code of '@' while it is loaded.

```
\int_new:N  \l__scontents_char_value_int
\int_set:Nn \l__scontents_char_value_int { \char_value_catcode:n { `\@ } }
\char_set_catcode_letter:N \@
\file_input:n { xparse-generic.tex }
\char_set_catcode:nn { `\@ } { \l__scontents_char_value_int }
⟨/!latex⟩
```

(*End of definition for* \c__scontents_version_str *and* \l__scontents_char_value_int.)

12.2.2 Checking expl3 version

For plain TEX, LATEX and ConTEXt we must check the minimum requirement, in this case `\tl_retokenize:n` which was added in release 2025-07-08 of `expl3` included in TEX Live 2025.

```
\cs_if_exist:NF \tl_retokenize:n
  {
    \msg_new:nnn { scontents } { expl-too-old }
      {
        Please~install~an~up~to~date~TeX~distribution~or~update~using~
        your~TeX~package~manager~or~from~CTAN. \\
        See~documentation.~Loading~scontents~will~abort!
      }
    \msg_fatal:nn { scontents } { expl-too-old }
    \ExplSyntaxOff
    \file_input_stop:
  }
```

12.2.3 Preventing double loading in plain TEX

In plain TEX, check that the package isn't being loaded twice, LATEX and ConTEXt already defend against that:

```
⟨*plain⟩
\tl_if_exist:NT \l__scontents_cmd_name_tl
  {
    \msg_new:nnn { scontents } { already-loaded }
      {
        The~'scontents'~package~is~already~loaded.~
        Aborting~input~\msg_line_context:.
      }
    \msg_warning:nn { scontents } { already-loaded }
    \ExplSyntaxOff
    \file_input_stop:
  }
⟨/plain⟩
⟨/loader⟩
```

12.2.4 Checking proper loader

Checking that the package was loaded with the proper loader code. This code was copied from `expl3-code.tex`.

```
⟨*core⟩
\def\ScontentsCoreFileDate{2026-11-01}%
\begingroup
  \catcode32=10
  \endlinechar=32
  \def\next{\endgroup}%
  \expandafter\ifx\csname PackageError\endcsname\relax
    \begingroup
      \def\next{\endgroup\endgroup}%
      \def\PackageError#1#2#3%
        {%
          \endgroup
          \errhelp{#3}%
          \errmessage{#1 Error: #2!}%
        }%
  \fi
  \expandafter\ifx\csname ScontentsFileDate\endcsname\relax
    \def\next
      {%
        \PackageError{scontents}{No scontents loader detected}
          {%
            You have attempted to use the scontents code directly rather
            than using the correct loader. Loading of scontents will abort.
          }%
        \endgroup
        \endinput
      }%
  \else
    \ifx\ScontentsFileDate\ScontentsCoreFileDate
    \else
      \def\next
```

```
        {%
          \PackageError{scontents}{Mismatched scontents files detected}
            {%
              You have attempted to load scontents with mismatched files:
              probably you have one or more files 'locally installed' which
              are in conflict. Loading of scontents will abort.
            }%
          \endgroup
          \endinput
        }%
    \fi
  \fi
\next
⟨/core⟩
```

12.3 Variables and functions by format

We define and set the variables and the function that must be handled separately in order to work properly with plain TEX, LATEX and ConTEXt.

__scontents_format_case:nnn

The function __scontents_format_case:nnn leaves the argument for the current format: #1 for LATEX, #2 for plain TEX and #3 for ConTEXt.

```
⟨*loader⟩
\cs_new:Npn \__scontents_format_case:nnn #1 #2 #3
⟨latex⟩   {#1} % LaTeX
⟨plain⟩   {#2} % Plain/Generic
⟨context⟩   {#3} % ConTeXt
```

(*End of definition for* __scontents_format_case:nnn.)

\c__scontents_newline_tl

The constant token list \c__scontents_newline_tl stores the character '^^J' used as end of line in the ⟨*stored content*⟩.

```
⟨*!context⟩
\tl_const:Ne \c__scontents_newline_tl { \iow_char:N \^^J }
⟨/!context⟩
```

In ConTEXt the package does not work under LMTX, so loading is aborted there. The test with \contextlmtxmode is adapted from the file t-lua-widow-control.mkxl of Max Chernoff's lua-widow-control package, \contextlmtxmode is described in https://source.contextgarden.net/tex/context/base/mkxl/context.mkxl.

```
⟨*context⟩
\bool_if:NTF \contextlmtxmode
  {
    \msg_new:nnn { scontents } { luametatex }
      {
        The~'scontents'~package~doesn't~work~under~LMTX.
      }
    \msg_fatal:nn { scontents } { luametatex }
  }
  {
    \tl_const:Ne \c__scontents_newline_tl { \iow_char:N \^^J }
  }
⟨/context⟩
```

(*End of definition for* \c__scontents_newline_tl.)

\g__scontents_end_verbatimsc_tl
\c__scontents_end_env_tl
\l__scontents_env_name_tl

The global token list \g__scontents_end_verbatimsc_tl matches the end of the verbatimsc environment (§12.14) in each format.

```
\tl_new:N \g__scontents_end_verbatimsc_tl
\tl_gset_rescan:Nnn \g__scontents_end_verbatimsc_tl
  {
    \char_set_catcode_other:N \\
⟨*latex⟩
    \char_set_catcode_other:N \{
    \char_set_catcode_other:N \}
⟨/latex⟩
  }
⟨latex⟩   { \end{verbatimsc} }
⟨plain⟩   { \endverbatimsc }
⟨context⟩   { \stopverbatimsc }
```

©2019–2026 by Pablo González L

The constant token list \c__scontents_end_env_tl matches the end of the environments defined by \newenvsc and the token list \l__scontents_env_name_tl stores the *name* of the environment being processed (§12.12).

```
\tl_new:N    \l__scontents_env_name_tl
\tl_const:Ne \c__scontents_end_env_tl
  {
    \c_backslash_str
⟨latex|plain⟩    end
⟨context⟩    stop
⟨latex⟩    \c_left_brace_str
      \exp_not:N \l__scontents_env_name_tl
⟨latex⟩    \c_right_brace_str
  }
```

(*End of definition for* \g__scontents_end_verbatimsc_tl, \c__scontents_end_env_tl, *and* \l__scontents_env_name_tl.)

12.4 Loading the package core

Now we load the core SCONTENTS code:

```
\file_input:n { scontents-code.tex }
⟨/loader⟩
```

12.5 Some kernel variants

\tl_if_empty:fTF
\tl_replace_all:NeV
\tl_retokenize:e

Some nonstandard kernel variants.

```
⟨*core⟩
\prg_generate_conditional_variant:Nnn \tl_if_empty:n { f } { p, TF }
\cs_generate_variant:Nn \tl_replace_all:Nnn { NeV }
\cs_generate_variant:Nn \tl_retokenize:n { e }
```

(*End of definition for* \tl_if_empty:fTF, \tl_replace_all:NeV, *and* \tl_retokenize:e.)

12.6 General variables

\q__scontents_stop
\q__scontents_mark
\s__scontents_stop
\s__scontents_mark

Quarks and scan marks used along the code as delimiters.

```
\quark_new:N \q__scontents_stop
\quark_new:N \q__scontents_mark
\scan_new:N  \s__scontents_stop
\scan_new:N  \s__scontents_mark
```

(*End of definition for* \q__scontents_stop *and others.*)

\l__scontents_cmd_name_tl

The token list \l__scontents_cmd_name_tl stores the *name* of the commands \Scontents, \foreachsc, \typestored, \meaningsc and \mergesc, it is used in the messages for unknown ⟨*keys*⟩ (§12.9).

```
\tl_new:N \l__scontents_cmd_name_tl
```

(*End of definition for* \l__scontents_cmd_name_tl.)

\l__scontents_file_name_tl
\l__scontents_file_write_iow
\l__scontents_writing_bool
\l__scontents_storing_bool
\l__scontents_writable_bool

The token list \l__scontents_file_name_tl stores the name of the ⟨*output file*⟩ set by the write-env, write-out and write-cmd keys and \l__scontents_file_write_iow is the *output stream* used to write it.

The boolean variables \l__scontents_writing_bool and \l__scontents_storing_bool, set by the same keys, determine whether the content is written to the ⟨*output file*⟩ and whether it is stored, \l__scontents_storing_bool is true by default. The boolean variable \l__scontents_writable_bool keeps track of whether the ⟨*output file*⟩ was opened by __scontents_file_if_writable:nTF (§12.10).

```
\tl_new:N   \l__scontents_file_name_tl
\iow_new:N  \l__scontents_file_write_iow
\bool_new:N \l__scontents_writing_bool
\bool_new:N \l__scontents_storing_bool
\bool_set_true:N \l__scontents_storing_bool
\bool_new:N \l__scontents_writable_bool
```

(*End of definition for* \l__scontents_file_name_tl *and others.*)

\l__scontents_seq_item_seq
\g__scontents_name_sc!internal_seq

The sequence \l__scontents_seq_item_seq stores the ⟨*indexes*⟩ of the ⟨*stored content*⟩ requested to \typestored, \meaningsc or \mergesc and the sequence \g__scontents_name_sc!internal_seq is the one assembled by \mergesc (§12.21).

```
\seq_new:N \l__scontents_seq_item_seq
\seq_new:c { g__scontents_name_sc!internal_seq }
```

(*End of definition for* \l__scontents_seq_item_seq *and* \g__scontents_name_sc!internal_seq.)

\g__scontents_last_stored_tl

The token list \g__scontents_last_stored_tl is used by the function __scontents_lastfrom_seq:n (§12.11) to execute the last ⟨*stored content*⟩ outside the group.

```
\tl_new:N \g__scontents_last_stored_tl
```

(*End of definition for* \g__scontents_last_stored_tl.)

\c__scontents_hidden_space_str

The constant string \c__scontents_hidden_space_str hides the ⟨*forced space*⟩ added by TEX when recording content in a macro. It contains the *reserved phrase* '%^^Ascheol%', which is added to the end of the ⟨*stored content*⟩ when the key force-eol is false.

```
\str_const:Ne \c__scontents_hidden_space_str
  { \c_percent_str \c_circumflex_str \c_circumflex_str A scheol \c_percent_str }
```

(*End of definition for* \c__scontents_hidden_space_str.)

\l__scontents_save_sf_int
\l__scontents_save_skip

Variables used by the functions __scontents_bsphack: and __scontents_esphack: (§12.10.2).

```
\int_new:N  \l__scontents_save_sf_int
\skip_new:N \l__scontents_save_skip
```

(*End of definition for* \l__scontents_save_sf_int *and* \l__scontents_save_skip.)

12.7 General messages

12.7.1 Definition of error messages

undefined-storage
index-out-of-range

The "error" undefined-storage is raised when the *sequence* passed to \getstored, \foreachsc, \typestored, \meaningsc, \mergesc or \countsc does not exist (§12.11, §12.17, §12.21, §12.23) and the "error" index-out-of-range is raised when the ⟨*index*⟩ is zero or its absolute value is greater than the number of ⟨*stored contents*⟩ (§12.11, §12.21).

```
\msg_new:nnnn { scontents } { undefined-storage }
  { Storage~named~'#1'~is~not~defined. }
  { No~content~has~been~stored~in~a~sequence~named~'#1'. }
\msg_new:nnnn { scontents } { index-out-of-range }
  {
    \int_compare:nNnTF {#1} = { 0 }
      { Index~of~sequence~'#2'~cannot~be~zero. }
      {
        Index~'#1'~out~of~range~for~'#2',~the~
        \int_compare:nNnTF {#1} > { 0 }
          { maximum~is~#3. }
          { minimum~is~-#3. }
      }
  }
  {
    The~sequence~'#2'~has~#3~stored~contents,~use~an~index~from~
    1~to~#3~or~from~-#3~to~-1.
  }
```

(*End of definition for* undefined-storage *and* index-out-of-range.)

positive-integer
empty-store-name

The "error" positive-integer is raised when the keys width-tab, start, step or stop receive a value that is not a ⟨*positive integer*⟩ (§12.8, §12.17.2) and the "error" empty-store-name is raised when the name of the *sequence* is blank (§12.11).

```
\msg_new:nnnn { scontents } { positive-integer }
  { The~key~'#1'~needs~a~positive~integer,~'#2'~given~\msg_line_context:. }
  { The~value~'#2'~is~ignored~and~the~key~'#1'~keeps~its~previous~value. }
\msg_new:nnnn { scontents } { empty-store-name }
  { Empty~sequence~name~for~storing~content~\msg_line_context:. }
  {
    The~keys~'store-env',~'store-cmd'~and~'store-all'~need~a~non-empty~
    sequence~name.~The~content~will~not~be~stored.
  }
```

(*End of definition for* positive-integer *and* empty-store-name.)

©2019–2026 by Pablo González L

12.7.2 Definition of messages for writing files

writing-file
overwrite-file
not-writing

The "info" messages `writing-file` and `overwrite-file` are written to the `.log` file when an ⟨*output file*⟩ is written or overwritten and the "warning" `not-writing` is issued when the ⟨*output file*⟩ already exists and the key `overwrite` is not active (§12.10).

```
\msg_new:nnn { scontents } { writing-file }
  { Writing~file~'#1'. }
\msg_new:nnn { scontents } { overwrite-file }
  { Overwriting~file~'#1'. }
\msg_new:nnn { scontents } { not-writing }
  { File~'#1'~already~exists.~Not~writing~\msg_line_context:. }
⟨/core⟩
```

(*End of definition for* `writing-file`, `overwrite-file`, *and* `not-writing`.)

12.8 Keys for the package

\l__scontents_tab_width_int
__scontents_set_positive_int:Nnn

The integer variable `\l__scontents_tab_width_int` stores the value of the `width-tab` key. The function `\__scontents_set_positive_int:Nnn` sets the integer variable passed in `#1` to the value passed in `#3` only if it is a ⟨*positive integer*⟩, otherwise it raises the "error" `positive-integer` naming the ⟨*key*⟩ passed in `#2` and the value is ignored. It is used by the `width-tab` key and by the `start`, `step` and `stop` keys of `\foreachsc` (§12.17.2).

```
⟨*core⟩
\int_new:N \l__scontents_tab_width_int
\cs_new_protected:Npn \__scontents_set_positive_int:Nnn #1 #2 #3
  {
    \bool_lazy_and:nnTF
      { ! \tl_if_blank_p:n {#3} }
      { \int_compare_p:nNn {#3} > { 0 } }
      { \int_set:Nn #1 {#3} }
      { \msg_error:nnnn { scontents } { positive-integer } {#2} {#3} }
  }
```

(*End of definition for* `\l__scontents_tab_width_int` *and* `\__scontents_set_positive_int:Nnn`.)

store-env
store-cmd
verb-font
print-env
print-cmd
force-eol
overwrite
width-tab
print-all
store-all
debug-var

We define the common ⟨*keys*⟩ used by the options passed to the package, by `\setupsc` and by the environments and commands.

```
\keys_define:nn { scontents }
  {
    store-env .tl_set:N         = \l__scontents_name_seq_env_tl,
    store-env .initial:n        = contents,
    store-env .value_required:n = true,
    store-cmd .tl_set:N         = \l__scontents_name_seq_cmd_tl,
    store-cmd .initial:n        = contents,
    store-cmd .value_required:n = true,
    verb-font .tl_set:N         = \l__scontents_verb_font_tl,
    verb-font .value_required:n = true,
    print-env .bool_set:N       = \l__scontents_print_env_bool,
    print-env .initial:n        = false,
    print-env .default:n        = true,
    print-cmd .bool_set:N       = \l__scontents_print_cmd_bool,
    print-cmd .initial:n        = false,
    print-cmd .default:n        = true,
    force-eol .bool_set:N       = \l__scontents_forced_eol_bool,
    force-eol .initial:n        = false,
    force-eol .default:n        = true,
    overwrite .bool_set:N       = \l__scontents_overwrite_bool,
    overwrite .initial:n        = false,
    overwrite .default:n        = true,
    width-tab .code:n           =
      { \__scontents_set_positive_int:Nnn \l__scontents_tab_width_int { width-tab } {#1} },
    width-tab .initial:n        = 1,
    width-tab .value_required:n = true,
    print-all .meta:n           = { print-env = #1, print-cmd = #1 },
    print-all .default:n        = true,
    store-all .meta:n           = { store-env = #1, store-cmd = #1 },
    store-all .value_required:n = true,
    debug-var .bool_set:N       = \l__scontents_debug_var_bool,
    debug-var .initial:n        = true,
    debug-var .default:n        = true,
```

©2019–2026 by Pablo González L

```
  }
⟨/core⟩
```

The default value of the `verb-font` key depends on the format.

```
⟨loader⟩\keys_define:nn { scontents }
⟨latex⟩  { verb-font .initial:n = \ttfamily }
⟨plain|context⟩  { verb-font .initial:n = \tt }
```

In LaTeX the ⟨*keys*⟩ passed as options to the package are processed and an unknown ⟨*key*⟩ returns an "error".

```
⟨*latex⟩
\ProcessKeyOptions [ scontents ]
⟨/latex⟩
```

(*End of definition for* `store-env` *and others.*)

12.9 Handling unknown keys

The ⟨*keys*⟩ that are not known by the environments and the commands are passed to the functions defined here. The name of the ⟨*key*⟩ is taken from `\l_keys_key_str` and the value, if any, is passed as argument.

12.9.1 Definition of error messages for unknown keys

`env-key-unknown`
`env-key-value-unknown`

The "errors" `env-key-unknown` and `env-key-value-unknown` are raised when an unknown ⟨*key*⟩, without or with value, is passed to an environment defined by `\newenvsc` (§12.12.4).

```
⟨*core⟩
\msg_new:nnnn { scontents } { env-key-unknown }
  {
    The~key~'#1'~is~unknown~by~environment~
    '\l__scontents_env_name_tl'~and~is~being~ignored.
  }
  {
    The~environment~'\l__scontents_env_name_tl'~does~not~have~a~key~called~'#1'.\\
    Check~that~you~have~spelled~the~key~name~correctly.
  }
\msg_new:nnnn { scontents } { env-key-value-unknown }
  {
    The~key~'#1=#2'~is~unknown~by~environment~
    '\l__scontents_env_name_tl'~and~is~being~ignored.
  }
  {
    The~environment~'\l__scontents_env_name_tl'~does~not~have~a~key~called~'#1'.\\
    Check~that~you~have~spelled~the~key~name~correctly.
  }
```

(*End of definition for* `env-key-unknown` *and* `env-key-value-unknown`.)

`cmd-key-unknown`
`cmd-key-value-unknown`

The "errors" `cmd-key-unknown` and `cmd-key-value-unknown` are raised when an unknown ⟨*key*⟩, without or with value, is passed to the commands `\Scontents`, `\foreachsc`, `\typestored`, `\meaningsc` or `\mergesc` (§12.15.3, §12.17.2, §12.18.2).

```
\msg_new:nnnn { scontents } { cmd-key-unknown }
  {
    The~key~'#1'~is~unknown~by~command~
    \c_backslash_str \l__scontents_cmd_name_tl \c_space_tl
    and~is~being~ignored~\msg_line_context:.
  }
  {
    The~command~\c_backslash_str \l__scontents_cmd_name_tl \c_space_tl
    does~not~have~a~key~called~'#1'.\\
    Check~that~you~have~spelled~the~key~name~correctly.
  }
\msg_new:nnnn { scontents } { cmd-key-value-unknown }
  {
    The~key~'#1=#2'~is~unknown~by~command~
    \c_backslash_str \l__scontents_cmd_name_tl \c_space_tl
    and~is~being~ignored~\msg_line_context:.
  }
  {
    The~command~\c_backslash_str \l__scontents_cmd_name_tl \c_space_tl
    does~not~have~a~key~called~'#1'.\\
    Check~that~you~have~spelled~the~key~name~correctly.
  }
```

©2019–2026 by Pablo González L

(*End of definition for* `cmd-key-unknown` *and* `cmd-key-value-unknown`.)

12.9.2 Internal functions for unknown keys

`\__scontents_unknown_keys_env:n`
`\__scontents_unknown_keys_env:nn`

The function `\__scontents_unknown_keys_env:n` raises the "error" for an unknown ⟨*key*⟩ passed to an environment defined by `\newenvsc`.

```
\cs_new_protected:Npn \__scontents_unknown_keys_env:n #1
  { \exp_args:NV \__scontents_unknown_keys_env:nn \l_keys_key_str {#1} }
\cs_new_protected:Npn \__scontents_unknown_keys_env:nn #1#2
  {
    \tl_if_blank:nTF {#2}
      { \msg_error:nnn { scontents } { env-key-unknown } {#1} }
      { \msg_error:nnnn { scontents } { env-key-value-unknown } {#1} {#2} }
  }
```

(*End of definition for* `\__scontents_unknown_keys_env:n` *and* `\__scontents_unknown_keys_env:nn`.)

`\__scontents_unknown_keys_cmd:n`
`\__scontents_unknown_keys_cmd:nn`

The function `\__scontents_unknown_keys_cmd:n` raises the "error" for an unknown ⟨*key*⟩ passed to the commands `\Scontents` and `\foreachsc`.

```
\cs_new_protected:Npn \__scontents_unknown_keys_cmd:n #1
  { \exp_args:NV \__scontents_unknown_keys_cmd:nn \l_keys_key_str {#1} }
\cs_new_protected:Npn \__scontents_unknown_keys_cmd:nn #1#2
  {
    \tl_if_blank:nTF {#2}
      { \msg_error:nnn { scontents } { cmd-key-unknown } {#1} }
      { \msg_error:nnnn { scontents } { cmd-key-value-unknown } {#1} {#2} }
  }
```

(*End of definition for* `\__scontents_unknown_keys_cmd:n` *and* `\__scontents_unknown_keys_cmd:nn`.)

12.10 Utility functions

`\__scontents_use_delimit_by_s_stop:nw`
`\__scontents_use_i_delimit_by_s_stop:nw`
`\__scontents_use_none_delimit_by_s_stop:w`
`\__scontents_use_none_delimit_by_q_stop:w`

Functions used in the implementation of `\newenvsc` (§12.12.5) and `\mergesc` (§12.21).

```
\cs_new:Npn \__scontents_use_delimit_by_s_stop:nw #1 \s__scontents_stop {#1}
\cs_new:Npn \__scontents_use_i_delimit_by_s_stop:nw #1 #2 \s__scontents_stop {#1}
\cs_new:Npn \__scontents_use_none_delimit_by_s_stop:w #1 \s__scontents_stop { }
\cs_new:Npn \__scontents_use_none_delimit_by_q_stop:w #1 \q__scontents_stop { }
```

(*End of definition for* `\__scontents_use_delimit_by_s_stop:nw` *and others.*)

`\__scontents_tl_if_head_is_q_mark:nTF`

The conditional `\__scontents_tl_if_head_is_q_mark:nTF` tests if the head of the token list is `\q__scontents_mark`.

```
\prg_new_protected_conditional:Npnn \__scontents_tl_if_head_is_q_mark:n #1
  { T, F, TF }
  {
    \exp_after:w \if_meaning:w
      \exp_after:w
        \q__scontents_mark \__scontents_use_i_delimit_by_s_stop:nw #1 ? \s__scontents_stop
      \prg_return_true:
    \else:
      \prg_return_false:
    \fi:
  }
```

(*End of definition for* `\__scontents_tl_if_head_is_q_mark:nTF`.)

`\__scontents_remove_trailing_eol:N`
`\__scontents_remove_trailing_eol:w`

The function `\__scontents_remove_trailing_eol:N` removes the string `\c__scontents_hidden_space_str` from the end of the token list passed in `#1`. It is used by `\typestored` (§12.19) and `\mergesc` (§12.21).

```
\cs_new_protected:Npe \__scontents_remove_trailing_eol:N #1
  {
    \exp_not:N \exp_after:w \exp_not:N \__scontents_remove_trailing_eol:w
      #1 \s__scontents_stop \c__scontents_hidden_space_str \s__scontents_stop \s__scontents_mark
  }
\use:e
  {
    \cs_new_protected:Npn \exp_not:N \__scontents_remove_trailing_eol:w #1
        \c__scontents_hidden_space_str \s__scontents_stop #2 \s__scontents_mark #3
  }  {
        \tl_set:Ne #3
```

```
            {
              \tl_if_empty:nTF {#2}
                { \exp_not:o { \__scontents_use_delimit_by_s_stop:nw #1 } }
                { \exp_not:n {#1} }
            }
        }
```

(*End of definition for* `\__scontents_remove_trailing_eol:N` *and* `\__scontents_remove_trailing_eol:w`.)

`\__scontents_file_if_writable:nTF` The conditional `\__scontents_file_if_writable:nTF` tests if the ⟨*output file*⟩ passed in `#1` can be written, according to the `write-env`, `write-cmd`, `write-out` and `overwrite` keys, and issues the messages for writing files (§12.7).

```
\prg_new_protected_conditional:Npnn \__scontents_file_if_writable:n #1 { T, F, TF }
  {
    \bool_if:NTF \l__scontents_writing_bool
      {
        \file_if_exist:nTF {#1}
          {
            \bool_if:NTF \l__scontents_overwrite_bool
              {
                \msg_info:nne { scontents } { overwrite-file } {#1}
                \prg_return_true:
              }
              {
                \msg_warning:nne { scontents } { not-writing } {#1}
                \prg_return_false:
              }
          }
          {
            \msg_info:nne { scontents } { writing-file } {#1}
            \prg_return_true:
          }
      }
      { \prg_return_false: }
  }
```

(*End of definition for* `\__scontents_file_if_writable:nTF`.)

`\__scontents_file_write_cmd:nn` `\__scontents_file_write_cmd:VV` The function `\__scontents_file_write_cmd:nn` writes the content passed in `#2` to the ⟨*output file*⟩ passed in `#1`, it is used by the commands.

```
\cs_new_protected:Npn \__scontents_file_write_cmd:nn #1#2
  {
    \__scontents_file_if_writable:nT {#1}
      {
        \iow_open:Nn \l__scontents_file_write_iow {#1}
        \iow_now:Nn  \l__scontents_file_write_iow {#2}
        \iow_close:N \l__scontents_file_write_iow
      }
  }
\cs_generate_variant:Nn \__scontents_file_write_cmd:nn { VV }
```

(*End of definition for* `\__scontents_file_write_cmd:nn`.)

12.10.1 Functions for TAB and verbatimsc

`\__scontents_tab:` `\__scontents_par:` Control sequences to replace the tab '`^^I`' and form feed '`^^L`' characters.

```
\cs_new:Ne \__scontents_tab: { \c_space_tl }
\cs_new:Nn \__scontents_par: { ^^J ^^J }
```

(*End of definition for* `\__scontents_tab:` *and* `\__scontents_par:`.)

`\__scontents_tabs_to_spaces:` In a *verbatim* context the TAB character is made active and set equal to `\__scontents_tabs_to_spaces:`, to produce as many spaces as the `width-tab` key was set to. The spaces are *active* characters, the same as the spaces of the *verbatim* context: a space token at the beginning of a line would be dropped in vertical mode and the TAB that indent a line would be lost. The *active* space is made with `\tex_lowercase:D` from the *active* character '`^^@`', since `\char_generate:nn` would expand it while defining the function.

```
\group_begin:
  \char_set_catcode_active:N \^^@
  \char_set_lccode:nn { 0 } { 32 }
```

```
  \tex_lowercase:D
    {
      \group_end:
      \cs_new:Npn \__scontents_tabs_to_spaces:
        { \prg_replicate:nn { \l__scontents_tab_width_int } { ^^@ } }
    }
```

(*End of definition for* `\__scontents_tabs_to_spaces:`.)

`\__scontents_do_noligs:N`

The function `\__scontents_do_noligs:N` is an alternative definition for LaTeX 2_ε's `\do@noligs` which makes sure to not consume the following space tokens. The LaTeX 2_ε version ends with `\char`#1`, which leaves TeX still looking for an ⟨*optional space*⟩.

```
\cs_new_protected:Npn \__scontents_do_noligs:N #1
  {
    \char_set_catcode_active:N #1
    \cs_set:cpe { __scontents_active_char_ \token_to_str:N #1 : }
      {
        \mode_leave_vertical:
        \tex_kern:D \c_zero_dim
        \tex_char:D `\exp_not:N #1
      }
    \char_set_active_eq:Nc #1 { __scontents_active_char_ \token_to_str:N #1 : }
  }
```

(*End of definition for* `\__scontents_do_noligs:N`.)

`\__scontents_set_active_eq:NN`
`\__scontents_plain_disable_outer_par:`
`\__scontents_make_control_chars_active:`

Shortcut definitions for common catcode changes. The '`^^L`' needs a special treatment in plain TeX because it is an `\outer` token there.

```
\cs_new_protected:Npn \__scontents_set_active_eq:NN #1
  {
    \char_set_catcode_active:N #1
    \char_set_active_eq:NN #1
  }
⟨/core⟩
⟨*loader⟩
\group_begin:
⟨plain⟩  \char_set_catcode_active:n { `\* }
  \cs_new_protected:Nn \__scontents_plain_disable_outer_par:
⟨*plain⟩
    {
      \group_begin:
        \char_set_lccode:nn { `\* } { `\^^L }
        \tex_lowercase:D { \group_end:
        \tex_let:D * \scan_stop:
      }
    }
⟨/plain⟩
⟨latex|context⟩    { }
\group_end:
⟨/loader⟩
⟨*core⟩
\cs_new_protected:Nn \__scontents_make_control_chars_active:
  {
    \__scontents_plain_disable_outer_par:
    \__scontents_set_active_eq:NN \^^I \__scontents_tab:
    \__scontents_set_active_eq:NN \^^L \__scontents_par:
    \__scontents_set_active_eq:NN \^^M \__scontents_ret:w
  }
```

(*End of definition for* `\__scontents_set_active_eq:NN`, `\__scontents_plain_disable_outer_par:`, *and* `\__scontents_make_control_chars_active:`.)

12.10.2 Functions `\@bsphack` and `\@esphack`

`\__scontents_bsphack:`
`\__scontents_esphack:`

We emulate `\@bsphack` and `\@esphack` for plain TeX and ConTeXt, this is necessary to prevent *unwanted spaces* when the `print-cmd` key is false. In LaTeX the kernel definitions are used.

```
\cs_new_protected:Nn \__scontents_bsphack:
  {
    \scan_stop:
    \mode_if_horizontal:T
```

```
      {
        \skip_set_eq:NN \l__scontents_save_skip \tex_lastskip:D
        \int_set_eq:NN \l__scontents_save_sf_int \tex_spacefactor:D
      }
  }
\cs_new_protected:Nn \__scontents_esphack:
  {
    \scan_stop:
    \mode_if_horizontal:T
      {
        \int_set_eq:NN \tex_spacefactor:D \l__scontents_save_sf_int
        \dim_compare:nNnT { \l__scontents_save_skip } > { \c_zero_skip }
          {
            \skip_if_eq:nnT { \tex_lastskip:D } { \c_zero_skip }
              {
                \nobreak
                \skip_horizontal:n { \c_zero_skip }
              }
            \tex_ignorespaces:D
          }
      }
  }
⟨/core⟩
⟨*latex⟩
\cs_gset_eq:NN \__scontents_bsphack: \@bsphack
\cs_gset_eq:NN \__scontents_esphack: \@esphack
⟨/latex⟩
```

(*End of definition for* `\__scontents_bsphack:` *and* `\__scontents_esphack:`.)

12.11 Functions for sequences

The *storage system* of the package SCONTENTS is done using `seq` variables. Here we set up the functions that manage them.

`\__scontents_append_contents:nn`
`\__scontents_append_contents:Ve`

The function `\__scontents_append_contents:nn` creates the *sequence* passed in `#1` if it does not exist and appends the {⟨*body env*⟩} of an environment or the {⟨*argument*⟩} of a command, passed in `#2`, to the right of the *sequence*.

```
⟨*core⟩
\cs_new_protected:Npn \__scontents_append_contents:nn #1#2
  {
    \seq_if_exist:cF { g__scontents_name_#1_seq }
      { \seq_new:c { g__scontents_name_#1_seq } }
    \seq_gput_right:cn { g__scontents_name_#1_seq } {#2}
  }
\cs_generate_variant:Nn \__scontents_append_contents:nn { Ve }
```

(*End of definition for* `\__scontents_append_contents:nn`.)

`\__scontents_store_to_seq:NN`

The function `\__scontents_store_to_seq:NN` writes the recorded content in `#1` to the `.log` file if `debug-var` is true and stores it in the *sequence* named by `#2`.

```
\cs_new_protected:Npn \__scontents_store_to_seq:NN #1#2
  {
    \bool_if:NT \l__scontents_debug_var_bool
      {
        \tl_log:N #1
      }
    \__scontents_append_contents:Ve #2 { \exp_not:V #1 }
  }
```

(*End of definition for* `\__scontents_store_to_seq:NN`.)

`\__scontents_finish_storing:NNN`

The function `\__scontents_finish_storing:NNN` works only in the *storing mode*, when the `write-out` key is not active. If the key `force-eol` is false, its default value, `\c__scontents_hidden_space_str` is added to the end of the token list passed in `#1`, which contains the {⟨*body env*⟩} of an environment or the {⟨*argument*⟩} of `\Scontents`.

Then `\__scontents_store_to_seq:NN` stores the content in the *sequence* named by `#2` and, if the boolean passed in `#3`, set by the `print-env` or `print-cmd` key, is true, the content is printed from

the *sequence* by __scontents_lastfrom_seq:V. If the *name* of the *sequence* is blank, the "error" empty-store-name is raised and nothing is stored or printed.

```
\cs_new_protected:Npn \__scontents_finish_storing:NNN #1 #2 #3
  {
    \bool_if:NT \l__scontents_storing_bool
      {
        \tl_if_blank:VTF #2
          { \msg_error:nn { scontents } { empty-store-name } }
          {
            \bool_if:NF \l__scontents_forced_eol_bool
              { \tl_put_right:Ne #1 { \c__scontents_hidden_space_str } }
            \__scontents_store_to_seq:NN #1 #2
            \bool_if:NT #3 { \__scontents_lastfrom_seq:V #2 }
          }
      }
  }
```

(*End of definition for* __scontents_finish_storing:NNN.)

__scontents_getfrom_seq:Nn
__scontents_getfrom_seq:nNn
__scontents_getfrom_seq_aux:nnn
__scontents_getfrom_seq:nn
__scontents_getfrom_seq:nnn

The function __scontents_getfrom_seq:Nn retrieves the ⟨*stored contents*⟩ whose ⟨*indexes*⟩ are in the sequence passed in #1 from the *sequence* named by #2 and the function __scontents_getfrom_seq:nn retrieves the one whose ⟨*index*⟩ is passed in #1. Both raise the "errors" undefined-storage and index-out-of-range (§12.7).

```
\cs_new:Npn \__scontents_getfrom_seq:Nn #1#2
  {
    \seq_if_exist:cTF { g__scontents_name_#2_seq }
      {
        \exp_args:Nf \__scontents_getfrom_seq:nNn
          { \seq_count:c { g__scontents_name_#2_seq } } #1 {#2}
      }
      { \msg_expandable_error:nnn { scontents } { undefined-storage } {#2} }
  }
\cs_new:Npn \__scontents_getfrom_seq:nNn #1 #2 #3
  {
    \seq_map_tokens:Nn #2 { \__scontents_getfrom_seq_aux:nnn {#1} {#3} }
  }
\cs_new:Npn \__scontents_getfrom_seq_aux:nnn #1 #2 #3
  {
    \exp_args:Nnf \use:n
      { \__scontents_getfrom_seq:nnn {#1} } { \int_eval:n {#3} } {#2}
  }
\cs_new:Npn \__scontents_getfrom_seq:nn #1#2
  {
    \seq_if_exist:cTF { g__scontents_name_#2_seq }
      {
        \exp_args:Nf \__scontents_getfrom_seq:nnn
          { \seq_count:c { g__scontents_name_#2_seq } }
          {#1} {#2}
      }
      { \msg_expandable_error:nnn { scontents } { undefined-storage } {#2} }
  }
\cs_new:Npn \__scontents_getfrom_seq:nnn #1#2#3
  {
    \bool_lazy_or:nnTF
      { \int_compare_p:nNn {#2} = { 0 } }
      { \int_compare_p:nNn { \int_abs:n {#2} } > {#1} }
      {
        \msg_expandable_error:nnnnn { scontents } { index-out-of-range }
          {#2} {#3} {#1}
      }
      { \seq_item:cn { g__scontents_name_#3_seq } {#2} }
  }
```

(*End of definition for* __scontents_getfrom_seq:Nn *and others.*)

__scontents_lastfrom_seq:n
__scontents_lastfrom_seq:V

The function __scontents_lastfrom_seq:n saves the last ⟨*stored content*⟩ of the *sequence* named by #1 in \g__scontents_last_stored_tl and rescans it with \tl_retokenize:V after the group, when the keys print-env or print-cmd are active.

```
\cs_new_protected:Npn \__scontents_lastfrom_seq:n #1
```

```
  {
    \tl_gset:Ne \g__scontents_last_stored_tl
      {
        \seq_item:cn { g__scontents_name_#1_seq } { -1 }
      }
    \group_insert_after:N \tl_retokenize:V
    \group_insert_after:N \g__scontents_last_stored_tl
    \group_insert_after:N \tl_gclear:N
    \group_insert_after:N \g__scontents_last_stored_tl
  }
\cs_generate_variant:Nn \__scontents_lastfrom_seq:n { V }
⟨/core⟩
```

(*End of definition for* `\__scontents_lastfrom_seq:n`.)

12.12 The command \newenvsc

The user-level command `\newenvsc` creates a new environment with the same behavior as the base environment `scontents`, which is itself defined with `\newenvsc` (§12.13). It takes a mandatory argument, the {⟨*env name*⟩}, and an optional argument with the ⟨*initial keys*⟩. The references to `\begin{scontents}` or `\end{scontents}` in this section apply to any environment defined with `\newenvsc`.

12.12.1 Definition of variables for \newenvsc

`\l__scontents_save_every_body_lines_tl`
`\l__scontents_processed_body_lines_tl`
`\l__scontents_environment_keys_tl`
`\l__scontents_nesting_env_int`
`\l__scontents_nesting_aux_int`

The token list `\l__scontents_save_every_body_lines_tl` holds the {⟨*body env*⟩} of the environment while it is being read and the token list `\l__scontents_processed_body_lines_tl` saves it once the environment is closed. The token list `\l__scontents_environment_keys_tl` saves the ⟨*keys*⟩ passed to the *optional argument* and the integer variables `\l__scontents_nesting_env_int` and `\l__scontents_nesting_aux_int` are used to analyze the nesting of the environment.

```
⟨*core⟩
\tl_new:N  \l__scontents_save_every_body_lines_tl
\tl_new:N  \l__scontents_processed_body_lines_tl
\tl_new:N  \l__scontents_environment_keys_tl
\int_new:N \l__scontents_nesting_env_int
\int_new:N \l__scontents_nesting_aux_int
```

(*End of definition for* `\l__scontents_save_every_body_lines_tl` *and others.*)

12.12.2 Definition of error messages for \newenvsc

`junk-after-begin`
`env-already-defined`

The "error" `junk-after-begin` is raised when there are characters after `\begin{scontents}` or its *optional argument* on the same line and the "error" `env-already-defined` is raised when `\newenvsc` is used with the name of an environment that already exists.

```
\msg_new:nnnn { scontents } { junk-after-begin }
  {
    Junk~characters~#1~\msg_line_context: :
    \\ \\
    #2
  }
  {
    Nothing~is~allowed~after~the~start~of~the~environment~on~the~same~
    line.~These~characters~are~ignored.
  }
\msg_new:nnnn { scontents } { env-already-defined }
  { Environment~'#1'~already~defined! }
  {
    You~have~used~\token_to_str:N \newenvsc \c_space_tl
    with~an~environment~that~already~has~a~definition. \\ \\
    The~existing~definition~of~'#1'~will~not~be~altered.
  }
```

(*End of definition for* `junk-after-begin` *and* `env-already-defined`.)

12.12.3 Definition of messages for \newenvsc

`empty-environment`
`rescanning-text`
`multiple-begin`

The "warning" `empty-environment` is issued when the environment is empty, the "warning" `rescanning-text` when there are characters after `\end{scontents}` on the same line and the "warning" `multiple-begin` when there is more than one `\begin{scontents}` on the same line.

```
\msg_new:nnn { scontents } { empty-environment }
  { Environment~'#1'~is~empty~\msg_line_context:. }
\msg_new:nnn { scontents } { rescanning-text }
```

©2019–2026 by Pablo González L

```
    { Rescanning~text~'#1'~after~\c_backslash_str end{#2}~\msg_line_context:. }
\msg_new:nnn { scontents } { multiple-begin }
    { Multiple~\c_backslash_str begin{ \l__scontents_env_name_tl }~\msg_line_context:. }
```

(*End of definition for* `empty-environment`, `rescanning-text`, *and* `multiple-begin`.)

12.12.4 Definition of keys for \newenvsc

write-env
write-out
print-env
store-env
force-eol
overwrite
unknown

We define the ⟨*keys*⟩ for the environments defined by \newenvsc, the unknown ⟨*keys*⟩ are passed to __scontents_unknown_keys_env:n (§12.9).

```
\keys_define:nn { scontents / scontents }
  {
    write-env .code:n           = {
                                    \bool_set_true:N \l__scontents_storing_bool
                                    \bool_set_true:N \l__scontents_writing_bool
                                    \tl_set:Nn \l__scontents_file_name_tl {#1}
                                  },
    write-out .code:n           = {
                                    \bool_set_false:N \l__scontents_storing_bool
                                    \bool_set_true:N  \l__scontents_writing_bool
                                    \tl_set:Nn \l__scontents_file_name_tl {#1}
                                  },
    write-env .value_required:n = true,
    write-out .value_required:n = true,
    print-env .meta:nn          = { scontents } { print-env = #1 },
    print-env .default:n        = true,
    store-env .meta:nn          = { scontents } { store-env = #1 },
    force-eol .meta:nn          = { scontents } { force-eol = #1 },
    force-eol .default:n        = true,
    overwrite .meta:nn          = { scontents } { overwrite = #1 },
    overwrite .default:n        = true,
    unknown   .code:n           = { \__scontents_unknown_keys_env:n {#1} },
  }
```

(*End of definition for* `write-env` *and others.*)

12.12.5 Internal functions for \newenvsc

__scontents_grab_opt_arg:w
__scontents_grab_opt_arg:n

The function __scontents_grab_opt_arg:w is called with the tokens following \begin{scontents} when the next character is a '['. It is defined with ltcmd[10] to use its delimited argument processor.

```
\NewDocumentCommand \__scontents_grab_opt_arg:w { r[] }
  {
    \__scontents_grab_opt_arg:n {#1}
  }
```

The function __scontents_grab_opt_arg:n is called where '^^M' is active, so __scontents_normalise_line_e replaces the active '^^M' characters by spaces before setting the ⟨*keys*⟩.

```
\cs_new_protected:Npn \__scontents_grab_opt_arg:n #1
  {
    \tl_set:Nn \l__scontents_environment_keys_tl {#1}
    \__scontents_normalise_line_ends:N \l__scontents_environment_keys_tl
    \keys_set:nV { scontents / scontents } \l__scontents_environment_keys_tl
    \__scontents_start_after_option:w
  }
```

(*End of definition for* `\__scontents_grab_opt_arg:w` *and* `\__scontents_grab_opt_arg:n`.)

__scontents_normalise_line_ends:N
__scontents_start_environment:w
__scontents_start_after_option:w
__scontents_check_line_process:nn
__scontents_stop_environment:

Here '^^I', '^^L' and '^^M' are active characters, so that the end of line can be used as a delimiter and TEX does not remove the space-like characters.

The function __scontents_start_environment:w checks if the next token after \begin{scontents} is a '[', in that case __scontents_grab_opt_arg:w processes the *optional argument* and calls __scontents_start_after_option:w. In all cases the function __scontents_check_line_process:nn raises the "error" `junk-after-begin` if there is something else on the line and then calls __scontents_file_tl_write_start:V, which reads the content of the environment and stores it in a token list or writes it to an ⟨*external file*⟩.

At the end of the environment, the function __scontents_stop_environment: calls __scontents_file_write_stop:N and issues the "warning" `empty-environment` if nothing was stored. This part of the code is adapted from the code of the package xsim[4] by Clemens Niederberger.

```
\group_begin:
```

©2019–2026 by Pablo González L

```
\char_set_catcode_active:N \^^I
\char_set_catcode_active:N \^^L
\char_set_catcode_active:N \^^M
\cs_new_protected:Npn \__scontents_normalise_line_ends:N #1
  { \tl_replace_all:Nnn #1 { ^^M } { ~ } }
\cs_new_protected:Npn \__scontents_start_environment:w #1 ^^M
  {
    \tl_if_head_is_N_type:nTF {#1}
      {
        \str_if_eq:eeTF { \tl_head:n {#1} } { [ }
          { \__scontents_grab_opt_arg:w #1 ^^M }
          { \__scontents_check_line_process:nn { } {#1} }
      }
      { \__scontents_check_line_process:nn { } {#1} }
  }
\cs_new_protected:Npn \__scontents_start_after_option:w #1 ^^M
  { \__scontents_check_line_process:nn { [...] } {#1} }
\cs_new_protected:Npn \__scontents_check_line_process:nn #1 #2
  {
    \tl_if_blank:nF {#2}
      {
        \msg_error:nnee { scontents } { junk-after-begin }
          { after~\c_backslash_str begin { \l__scontents_env_name_tl } #1 } {#2}
      }
    \__scontents_make_control_chars_active:
    \__scontents_file_tl_write_start:V \l__scontents_file_name_tl
  }
\cs_new_protected:Nn \__scontents_stop_environment:
  {
    \__scontents_file_write_stop:N \l__scontents_processed_body_lines_tl
    \bool_lazy_and:nnT
      { \l__scontents_storing_bool }
      { \tl_if_empty_p:N \l__scontents_processed_body_lines_tl }
      {
        \msg_warning:nne { scontents } { empty-environment }
          { \l__scontents_env_name_tl }
      }
  }
```

(*End of definition for* `\__scontents_normalise_line_ends:N` *and others.*)

`\__scontents_file_tl_write_start:n`
`\__scontents_file_tl_write_start:V`
`\__scontents_setup_verb_processor:`
`\__scontents_verb_processor_iterate:w`
`\__scontents_verb_processor_iterate:nnn`
`\__scontents_env_end_function:`
`\__scontents_file_write_stop:N`
`\__scontents_remove_leading_nl:n`
`\__scontents_remove_leading_nl:nn`
`\__scontents_remove_leading_nl:w`

This is the main function to collect the {⟨*body env*⟩} of the environment. The function `\__scontents_file_tl_write_start:n` starts a *group*, opens the ⟨*output file*⟩ if necessary, sets the *verbatim catcodes* and then issues '`^^M`', set equal to `\__scontents_ret:w`, to read the {⟨*body env*⟩} line by line until its end. Each line read is appended to the output token list after an active '`^^J`' and written to the ⟨*output file*⟩, if any.

The function `\__scontents_setup_verb_processor:` defines `\__scontents_verb_processor_iterate:w`, delimited by the end of the current environment, and `\__scontents_verb_processor_iterate:nnn` decides if the line is a nested `\end{scontents}` or the end of the environment, in which case `\__scontents_env_end_function:` leaves the closing command for the current format and the characters after it are rescanned with the "warning" `rescanning-text`.

At the end of the environment, the function `\__scontents_file_write_stop:N` closes the ⟨*output file*⟩, if it was opened, and smuggles the output token list out of the group. The leading '`^^J`' is removed with `\__scontents_remove_leading_nl:n`, which expects an active '`^^J`' at the head of the token list, otherwise a low level TEX "error" is raised.

```
\cs_new_protected:Npn \__scontents_file_tl_write_start:n #1
  {
    \group_begin:
      \__scontents_file_if_writable:nTF {#1}
        {
          \bool_set_true:N \l__scontents_writable_bool
          \iow_open:Nn \l__scontents_file_write_iow {#1}
        }
        { \bool_set_false:N \l__scontents_writable_bool }
      \tl_clear:N \l__scontents_save_every_body_lines_tl
      \seq_map_function:NN \l_char_special_seq \char_set_catcode_other:N
      \int_step_function:nnnN { 128 } { 1 } { 255 } \char_set_catcode_letter:n
      \cs_set_protected:Npe \__scontents_ret:w ##1 ^^M
        {
```

©2019–2026 by Pablo González L

```
            \exp_not:N \__scontents_verb_processor_iterate:w
            ##1 \c__scontents_end_env_tl
                \c__scontents_end_env_tl
                \exp_not:N \q__scontents_stop
          }
        \__scontents_make_control_chars_active:
        \__scontents_ret:w
    }
  \cs_new_protected:Nn \__scontents_setup_verb_processor:
    {
      \use:e
        {
          \cs_set_protected:Npn \exp_not:N \__scontents_verb_processor_iterate:w
            ##1 \c__scontents_end_env_tl
            ##2 \c__scontents_end_env_tl
            ##3 \exp_not:N \q__scontents_stop
        }   { \__scontents_verb_processor_iterate:nnn {##1} {##2} {##3} }
    }
  \cs_new_protected:Npn \__scontents_verb_processor_iterate:nnn #1 #2 #3
    {
      \tl_if_blank:nTF {#3}
        {
          \__scontents_analyse_nesting:n {#1}
          \__scontents_verb_processor_output:n {#1}
        }
        {
          \__scontents_if_nested:TF
            {
              \__scontents_nesting_decr:
              \__scontents_verb_processor_output:e
                { \exp_not:n {#1} \c__scontents_end_env_tl \exp_not:n {#2} }
            }
            {
              \tl_if_blank:nF {#1}
                { \__scontents_verb_processor_output:n {#1} }
              \cs_set_protected:Npe \__scontents_ret:w
                {
                  \__scontents_env_end_function:
                  \bool_lazy_or:nnF
                    { \tl_if_blank_p:n {#2} }
                    { \str_if_eq_p:ee {#2} { \c_percent_str } }
                    {
                      \str_if_eq:VnF \c__scontents_hidden_space_str {#2}
                        {
                          \msg_warning:nnnn { scontents } { rescanning-text }
                            {#2} { \tl_use:N \l__scontents_env_name_tl }
                        }
                      \tl_retokenize:n {#2}
                    }
                }
              \char_set_active_eq:NN ^^M \__scontents_ret:w
            }
        }
      ^^M
    }
  \cs_new:Nn \__scontents_env_end_function:
    {
      \__scontents_format_case:nnn
        { \exp_not:N \end { \if_false: } \fi: }
        { \exp_after:w \exp_not:N \cs:w end }
        { \exp_after:w \exp_not:N \cs:w stop }
      \tl_use:N \l__scontents_env_name_tl
      \__scontents_format_case:nnn
        { \if_false: { \fi: } }
        { \cs_end: }
        { \cs_end: }
    }
  \cs_new_protected:Npn \__scontents_file_write_stop:N #1
    {
      \bool_if:NT \l__scontents_writable_bool
        { \iow_close:N \l__scontents_file_write_iow }
```

```
      \use:e
        {
          \group_end:
          \bool_if:NT \l__scontents_storing_bool
            {
              \tl_set:Nn \exp_not:N #1
                {
                  \exp_args:NV
                    \__scontents_remove_leading_nl:n \l__scontents_save_every_body_lines_tl
                }
            }
        }
    }
  \cs_new:Npn \__scontents_remove_leading_nl:n #1
    {
      \tl_if_head_is_N_type:nTF {#1}
        {
          \exp_args:Nf
            \__scontents_remove_leading_nl:nn
              { \tl_head:n {#1} } {#1}
        }
        { \exp_not:n {#1} }
    }
  \cs_new:Npn \__scontents_remove_leading_nl:nn #1 #2
    {
      \token_if_eq_meaning:NNTF ^^J #1
        { \exp_not:o { \__scontents_remove_leading_nl:w #2 } }
        { \exp_not:n {#2} }
    }
  \cs_new:Npn \__scontents_remove_leading_nl:w ^^J { }
```

(*End of definition for* `\__scontents_file_tl_write_start:n` *and others.*)

`\__scontents_verb_processor_output:n`
`\__scontents_verb_processor_output:e`

The function `\__scontents_verb_processor_output:n` does the output of each line read, to the token list and to the ⟨*output file*⟩, according to the booleans `\l__scontents_writable_bool` and `\l__scontents_storing_bool`.

```
  \cs_new_protected:Npn \__scontents_verb_processor_output:n #1
    {
      \bool_if:NT \l__scontents_writable_bool
        { \iow_now:Nn \l__scontents_file_write_iow {#1} }
      \bool_if:NT \l__scontents_storing_bool
        { \tl_put_right:Nn \l__scontents_save_every_body_lines_tl { ^^J #1 } }
    }
\group_end:
\cs_generate_variant:Nn \__scontents_verb_processor_output:n { e }
\cs_generate_variant:Nn \__scontents_file_tl_write_start:n { V }
```

(*End of definition for* `\__scontents_verb_processor_output:n`.)

`\__scontents_analyse_nesting:n`
`\__scontents_nesting_incr:`
`\__scontents_nesting_decr:`
`\__scontents_if_nested:TF`

The function `\__scontents_analyse_nesting:n` scans the nested `\begin{scontents}` in a line and steps the counter `\l__scontents_nesting_env_int`, issuing the "warning" `multiple-begin` if there is more than one. The conditional `\__scontents_if_nested:TF` tests if we are in a nested environment and `\__scontents_nesting_decr:` reduces the nesting level when an `\end{scontents}` is found.

```
\cs_new_protected:Npn \__scontents_analyse_nesting:n #1
  {
    \int_zero:N \l__scontents_nesting_aux_int
    \__scontents_analyse_nesting_format:n {#1}
    \int_compare:nNnT { \l__scontents_nesting_aux_int } > { 1 }
      { \msg_warning:nn { scontents } { multiple-begin } }
  }
\cs_new_protected:Nn \__scontents_nesting_incr:
  {
    \int_incr:N \l__scontents_nesting_env_int
    \int_incr:N \l__scontents_nesting_aux_int
  }
\cs_new_protected:Nn \__scontents_nesting_decr:
  {
    \int_decr:N \l__scontents_nesting_env_int
  }
\prg_new_protected_conditional:Npnn \__scontents_if_nested: { TF }
```

©2019–2026 by Pablo González L

```
  {
    \int_compare:nNnTF { \l__scontents_nesting_env_int } > { \c_zero_int }
      { \prg_return_true: }
      { \prg_return_false: }
  }
```

(*End of definition for* `\__scontents_analyse_nesting:n` *and others.*)

Multiple `\end{scontents}` on the same line are NOT supported.

`\__scontents_analyse_nesting_latex:n`
`\__scontents_analyse_nesting_latex:w`
`\__scontents_analyse_nesting_generic:nn`
`\__scontents_analyse_nesting_generic:w`
`ontents_analyse_nesting_generic_process:nn`
`contents_define_generic_nesting_function:n`
`\__scontents_analyse_nesting_format:n`

In LaTeX the environments start with `\begin{«env»}` and no '}' can appear inside «env», so a macro delimited by '}' is enough to find the nested `\begin{scontents}`.

```
\use:e
  {
    \cs_new_protected:Npn \exp_not:N \__scontents_analyse_nesting_latex:w #1
      \c_backslash_str begin \c_left_brace_str #2 \c_right_brace_str
  }  {
      \__scontents_tl_if_head_is_q_mark:nTF {#2}
        { \__scontents_use_none_delimit_by_q_stop:w }
        {
          \str_if_eq:VnT \l__scontents_env_name_tl {#2}
            { \__scontents_nesting_incr: }
          \__scontents_analyse_nesting_latex:w
        }
    }
\cs_new_protected:Npe \__scontents_analyse_nesting_latex:n #1
  {
    \__scontents_analyse_nesting_latex:w #1
      \c_backslash_str begin
        \c_left_brace_str \exp_not:N \q__scontents_mark \c_right_brace_str
    \exp_not:N \q__scontents_stop
  }
```

In the other formats there is no "end anchor" to delimit the name of the environment, so we search for the whole command that starts the environment, `\scontents` or `\startscontents`.

```
\cs_new_protected:Npn \__scontents_analyse_nesting_generic_process:nn #1 #2
  {
    \tl_if_head_is_N_type:nTF {#2}
      {
        \__scontents_tl_if_head_is_q_mark:nF {#2}
          {
            \__scontents_nesting_incr:
            \__scontents_analyse_nesting_generic:w #2 \q__scontents_stop
          }
      }
      { \__scontents_analyse_nesting_generic:w #2 \q__scontents_stop }
  }
\cs_new_protected:Npn \__scontents_analyse_nesting_generic:nn #1 #2
  {
    \__scontents_define_generic_nesting_function:n {#1}
    \use:e
      {
        \exp_not:N \__scontents_analyse_nesting_generic:w #2
          \c_backslash_str #1 \tl_use:N \l__scontents_env_name_tl
            \exp_not:N \q__scontents_mark \exp_not:N \q__scontents_stop
      }
  }
\cs_new_protected:Npn \__scontents_define_generic_nesting_function:n #1
  {
    \use:e
      {
        \cs_set_protected:Npn \exp_not:N \__scontents_analyse_nesting_generic:w ##1
          \c_backslash_str #1 \tl_use:N \l__scontents_env_name_tl
            ##2 \exp_not:N \q__scontents_stop
      }   { \__scontents_analyse_nesting_generic_process:nn {##1} {##2} }
  }
⟨/core⟩
```

The function `\__scontents_analyse_nesting_format:n` uses the version for the current format.

```
⟨*loader⟩
⟨latex⟩\cs_new_eq:NN \__scontents_analyse_nesting_format:n
```

©2019–2026 by Pablo González L

```
⟨latex⟩  \__scontents_analyse_nesting_latex:n
⟨!latex⟩\cs_new_protected:Npn \__scontents_analyse_nesting_format:n
⟨plain⟩  { \__scontents_analyse_nesting_generic:nn { } }
⟨context⟩  { \__scontents_analyse_nesting_generic:nn { start } }
⟨/loader⟩
```

(*End of definition for* `\__scontents_analyse_nesting_latex:n` *and others.*)

`\__scontents_generic_begin:`
`\__scontents_generic_end:`

The function `\__scontents_generic_begin:` leaves the '`^^M`' character active and calls `\__scontents_start_environment:w`. The function `\__scontents_generic_end:` calls `\__scontents_stop_environment:` and then `\__scontents_finish_storing:NNN`, which stores the {⟨*body env*⟩} in the *sequence* and prints it if the `print-env` key is active (§12.11).

```
⟨*core⟩
\cs_new_protected:Npn \__scontents_generic_begin:
  {
    \char_set_catcode_active:N \^^M
    \__scontents_start_environment:w
  }
\cs_new_protected:Npn \__scontents_generic_end:
  {
    \__scontents_stop_environment:
    \__scontents_finish_storing:NNN
      \l__scontents_processed_body_lines_tl
      \l__scontents_name_seq_env_tl
      \l__scontents_print_env_bool
  }
```

(*End of definition for* `\__scontents_generic_begin:` *and* `\__scontents_generic_end:`.)

`\__scontents_setting_env:nn`
`\__scontents_define_env:nnn`

The function `\__scontents_setting_env:nn` receives the *name* of the environment in `#1` and the ⟨*initial keys*⟩ in `#2`. It creates the functions `\__scontents_#1_begin:` and `\__scontents_#1_end:`, which set `\l__scontents_env_name_tl` and the ⟨*initial keys*⟩ and call `\__scontents_generic_begin:` and `\__scontents_generic_end:`, and passes them to `\__scontents_define_env:nnn`.

```
\cs_new_protected:Npn \__scontents_setting_env:nn #1 #2
  {
    \cs_new_protected:cpn { __scontents_#1_begin: }
      {
        \tl_set:Nn \l__scontents_env_name_tl {#1}
        \keys_set:nn { scontents } {#2}
        \__scontents_setup_verb_processor:
        \__scontents_generic_begin:
      }
    \cs_new_protected:cpn { __scontents_#1_end: }
      { \__scontents_generic_end: }
    \exp_args:Nooo \__scontents_define_env:nnn
      { \tl_to_str:n {#1} }
      { \cs:w __scontents_#1_begin: \cs_end: }
      { \cs:w __scontents_#1_end: \cs_end: }
  }
⟨/core⟩
```

The function `\__scontents_define_env:nnn` creates the environment for LaTeX, plain TeX and ConTeXt.

```
⟨*loader⟩
\cs_new_protected:Npn \__scontents_define_env:nnn #1 #2 #3
  {
⟨latex|plain⟩    \NewDocumentEnvironment {#1} { }
⟨context⟩    \cs_new_protected:cpn { start #1 }
      {
⟨!latex⟩        \group_begin:
          #2
      }
⟨context⟩    \cs_new_protected:cpn { stop #1 }
      {
          #3
⟨!latex⟩        \group_end:
      }
  }
```

(*End of definition for* `\__scontents_setting_env:nn` *and* `\__scontents_define_env:nnn`.)

©2019–2026 by Pablo González L

\newenvsc Defines the user command \newenvsc for LaTeX, plain TeX and ConTeXt, raising the "error" env-already-defined if the environment exists.

```
\NewDocumentCommand \newenvsc { m O{} }
  {
⟨latex|plain⟩    \cs_if_exist:cTF { #1 }
⟨context⟩    \cs_if_exist:cTF { start #1 }
      { \msg_error:nnn { scontents } { env-already-defined } {#1} }
      { \__scontents_setting_env:nn {#1} {#2} }
  }
⟨/loader⟩
```

(*End of definition for* \newenvsc*. This function is documented on page 5.*)

12.13 The environment scontents

scontents \scontents \endscontents \startscontents \stopscontents The base environment scontents is defined with \newenvsc, in plain TeX it is \scontents ... \endscontents and in ConTeXt \startscontents ... \stopscontents.

```
⟨*loader⟩
\newenvsc{scontents}
⟨/loader⟩
```

(*End of definition for* scontents *and others. These functions are documented on page 4.*)

12.14 The environment verbatimsc

The verbatimsc environment is a customized version of the standard verbatim environment of LaTeX used by \typestored (§12.19). For plain TeX and ConTeXt we must add some additional functions.

12.14.1 Definition of variables for verbatimsc

\l__scontents_plain_bool The boolean variable \l__scontents_plain_bool is used in the plain TeX version of the verbatimsc environment.

```
⟨*plain⟩
\bool_new:N \l__scontents_plain_bool
⟨/plain⟩
```

(*End of definition for* \l__scontents_plain_bool*.*)

12.14.2 Internal functions for verbatimsc

\dospecials The verbatim environment requires \dospecials. In case it does not exist when SCONTENTS is loaded, we define it with \l_char_special_seq.

```
⟨*!latex⟩
\cs_if_exist:NF \dospecials
  {
    \cs_new:Npn \dospecials
      { \seq_map_function:NN \l_char_special_seq \do }
  }
⟨/!latex⟩
```

(*End of definition for* \dospecials*.*)

__scontents_xverb:w __scontents_xverb: The environment verbatimsc needs to find literally its end, this is done by __scontents_xverb:w, delimited by \g__scontents_end_verbatimsc_tl, for plain TeX and LaTeX. The function __scontents_xverb: makes the TAB character active, equal to __scontents_tabs_to_spaces:, and calls __scontents_xverb:w.

```
⟨*loader⟩
⟨*!context⟩
\use:e
  {
    \cs_new_protected:Npn \exp_not:N \__scontents_xverb:w
      #1 \g__scontents_end_verbatimsc_tl
⟨latex⟩      { #1 \exp_not:N \end{verbatimsc} }
⟨plain⟩      { #1 \exp_not:N \endverbatimsc }
  }
⟨/!context⟩
⟨/loader⟩
⟨*core⟩
\cs_new_protected:Nn \__scontents_xverb:
  {
```

```
    \char_set_catcode_active:n { 9 }
    \char_set_active_eq:nN { 9 } \__scontents_tabs_to_spaces:
    \__scontents_xverb:w
  }
⟨/core⟩
```

(*End of definition for* `\__scontents_xverb:w` *and* `\__scontents_xverb:`.)

12.14.3 plain TEX version of verbatimsc

`\__scontents_verbatimsc_aux:` `\__scontents_nolig_list:` `\__scontents_vobeyspaces:` `\__scontents_xobeysp:`

In plain TEX we emulate the verbatim environment of LATEX.

```
⟨*plain⟩
\cs_new_protected:Nn \__scontents_verbatimsc_aux:
  {
    \skip_vertical:N \parskip
    \dim_zero:N \parindent
    \skip_set:Nn \parfillskip { 0pt plus 1fil }
    \skip_set:Nn \parskip { 0pt plus0pt minus0pt }
    \tex_par:D
    \bool_set_false:N \l__scontents_plain_bool
    \cs_set:Npn \par
      {
        \bool_if:NTF \l__scontents_plain_bool
          {
            \mode_leave_vertical:
            \null
            \tex_par:D
            \penalty \interlinepenalty
          }
          {
            \bool_set_true:N \l__scontents_plain_bool
            \mode_if_horizontal:T
              { \tex_par:D \penalty \interlinepenalty }
          }
      }
    \cs_set_eq:NN \do \char_set_catcode_other:N
    \dospecials \obeylines
    \tl_use:N \l__scontents_verb_font_tl
    \cs_set_eq:NN \do \__scontents_do_noligs:N
    \__scontents_nolig_list:
    \tex_everypar:D \exp_after:w
      { \tex_the:D \tex_everypar:D \tex_unpenalty:D }
  }
\cs_new_protected:Nn \__scontents_nolig_list:
  { \do\`\do\<\do\>\do\,\do\'\do\- }
\cs_new_protected:Nn \__scontents_vobeyspaces:
  { \__scontents_set_active_eq:NN \  \__scontents_xobeysp: }
\cs_new_protected:Nn \__scontents_xobeysp:
  { \mode_leave_vertical: \nobreak \ }
```

(*End of definition for* `\__scontents_verbatimsc_aux:` *and others.*)

`\verbatimsc` `\endverbatimsc`

Defines the environment verbatimsc for plain TEX.

```
\cs_new_protected:Npn \verbatimsc
  {
    \group_begin:
      \__scontents_verbatimsc_aux: \frenchspacing \__scontents_vobeyspaces:
      \__scontents_xverb:
  }
\cs_new_protected:Npn \endverbatimsc
  { \group_end: }
⟨/plain⟩
```

(*End of definition for* `\verbatimsc` *and* `\endverbatimsc`. *These functions are documented on page* 7.)

12.14.4 ConTEXt version of verbatimsc

`\startverbatimsc` `\stopverbatimsc`

In ConTEXt we use its own tool \definetyping.

```
⟨*loader⟩
⟨context⟩\definetyping[verbatimsc]
⟨/loader⟩
```

(*End of definition for* `\startverbatimsc` *and* `\stopverbatimsc`. *These functions are documented on page* ??.)

©2019–2026 by Pablo González L

12.14.5 LaTeX version of verbatimsc

__scontents_verbatimsc_instance:

To be compatible with *tagged* PDF the environment verbatimsc is defined with the xtemplate[10] module of the LaTeX kernel and the blocks-code of latex-lab[13]. The function __scontents_verbatimsc_instance: declares the instance of blockenv used by the environment. This code is adapted from the answer of Ulrike Fischer to New verbatim environment with block code (tagged-pdf).

```
⟨*loader⟩
⟨*latex⟩
\cs_new_protected:Nn \__scontents_verbatimsc_instance:
  {
    \DeclareInstance{blockenv}{verbatimsc}{std}
      {
        name           = verbatimsc,
        tag-name       = \UseStructureName{block/verbatim},
        tagging-recipe = standard,
        tagging-suppress-paras = true,
        block-instance = verbatim,
        para-instance  = justify ,
        final-code     = \legacyverbatimsetup{invisible},
      }
  }
```

(*End of definition for* __scontents_verbatimsc_instance:.)

verbatimsc

Defines the environment verbatimsc for LaTeX. The \endverbatim in the second argument is only needed for compatibility with the verbatim[15] package.

```
\NewDocumentEnvironment { verbatimsc } { }
  {
    \IfDocumentMetadataTF
      {
        \__scontents_verbatimsc_instance:
        \UseInstance{blockenv}{verbatimsc}{}
        \@setupverbinvisiblespace\frenchspacing\@vobeyspaces
        \__scontents_xverb:
      }
      {
        \cs_set_eq:cN { @xverbatim } \__scontents_xverb:
        \verbatim
      }
  }
  {
    \IfDocumentMetadataTF
      {
        \BlockEnvEnd
      }
      { \endverbatim }
  }
⟨/latex⟩
⟨/loader⟩
```

(*End of definition for* verbatimsc. *This function is documented on page* 7.)

12.15 The command \Scontents

The user-level command \Scontents stores its {⟨*argument*⟩} in a *sequence*. It takes an optional *starred argument*, an optional argument with the ⟨*keys*⟩ and the {⟨*argument*⟩}, which is read in standard mode or, with the *starred argument*, under the *verbatim catcode* regimen. It is adapted from code by Ulrich Diez in Stringify input - \string on token list and code by user @siracusa in Convert a macro from Latex2e to expl3.

12.15.1 Definition of variables for \Scontents

\l__scontents_Scontents_arg_tl

The token list \l__scontents_Scontents_arg_tl saves the {⟨*argument*⟩} passed to \Scontents.

```
⟨*core⟩
\tl_new:N \l__scontents_Scontents_arg_tl
```

(*End of definition for* \l__scontents_Scontents_arg_tl.)

©2019–2026 by Pablo González L

12.15.2 Definition of messages for \Scontents

empty-argument-command

The "warning" `empty-argument-command` is issued when the {⟨*argument*⟩} of `\Scontents` is empty.

```
\msg_new:nnn { scontents } { empty-argument-command }
  { Empty~argument~for~\c_backslash_str Scontents~\msg_line_context:. }
```

(*End of definition for* `empty-argument-command`.)

12.15.3 Definition of keys for \Scontents

write-cmd
write-out
print-cmd
store-cmd
force-eol
overwrite
unknown

We define the ⟨*keys*⟩ for both versions of `\Scontents`, the unknown ⟨*keys*⟩ are passed to `\__scontents_unknown_keys_cmd:n` (§12.9).

```
\keys_define:nn { scontents / Scontents }
  {
    write-cmd .code:n           = {
                                    \bool_set_true:N \l__scontents_storing_bool
                                    \bool_set_true:N \l__scontents_writing_bool
                                    \tl_set:Nn \l__scontents_file_name_tl {#1}
                                  },
    write-out .code:n           = {
                                    \bool_set_false:N \l__scontents_storing_bool
                                    \bool_set_true:N  \l__scontents_writing_bool
                                    \tl_set:Nn \l__scontents_file_name_tl {#1}
                                  },
    write-cmd .value_required:n = true,
    write-out .value_required:n = true,
    print-cmd .meta:nn          = { scontents } { print-cmd = #1 },
    print-cmd .default:n        = true,
    store-cmd .meta:nn          = { scontents } { store-cmd = #1 },
    force-eol .meta:nn          = { scontents } { force-eol = #1 },
    force-eol .default:n        = true,
    overwrite .meta:nn          = { scontents } { overwrite = #1 },
    overwrite .default:n        = true,
    unknown   .code:n           = { \__scontents_unknown_keys_cmd:n {#1} },
  }
```

(*End of definition for* `write-cmd` *and others.*)

12.15.4 Internal functions for \Scontents

__scontents_Scontents_code:nn

The function `\__scontents_Scontents_code:nn` executes `\__scontents_bsphack:`, opens a group, sets the ⟨*keys*⟩ passed in `#2` and leaves the `TAB` character active. With the *starred argument* passed in `#1` it calls `\__scontents_Scontents_verb_arg:w`, otherwise `\__scontents_Scontents_norm_arg:n`.

```
\cs_new_protected:Npn \__scontents_Scontents_code:nn #1 #2
  {
    \__scontents_bsphack:
    \group_begin:
      \keys_set:nn { scontents / Scontents } {#2}
      \char_set_catcode_active:n { 9 }
      \bool_if:NTF #1
        { \__scontents_Scontents_verb_arg:w }
        { \__scontents_Scontents_norm_arg:n }
  }
```

(*End of definition for* `\__scontents_Scontents_code:nn`.)

__scontents_Scontents_verb_arg:w

The function `\__scontents_Scontents_verb_arg:w` saves the {⟨*argument*⟩} under the *verbatim catcode* regimen, using a v-type argument of `ltcmd`, in `\l__scontents_Scontents_arg_tl` and replaces every '`^^M`' and `\obeyedline`, added in LaTeX release 2024-06-01, by '`^^J`'. The command `\obeyedline` is redefined with `\RenewDocumentCommand` because it can be modified by the user, which would return a low-level error.

```
\NewDocumentCommand \__scontents_Scontents_verb_arg:w { +v }
  {
    \tl_set:Nn \l__scontents_Scontents_arg_tl {#1}
    \cs_if_exist:NT \obeyedline
      {
        \RenewDocumentCommand \obeyedline { } { \iow_char:N \^^M }
        \tl_replace_all:Nee \l__scontents_Scontents_arg_tl
          { \obeyedline } { \iow_char:N \^^M }
      }
```

©2019–2026 by Pablo González L

```
    \tl_replace_all:NeV \l__scontents_Scontents_arg_tl
      { \iow_char:N \^^M } \c__scontents_newline_tl
    \__scontents_Scontents_finish:
  }
```

(*End of definition for* `\__scontents_Scontents_verb_arg:w`.)

`\__scontents_Scontents_norm_arg:n` The function `\__scontents_Scontents_norm_arg:n` saves the {⟨*argument*⟩} read in the *standard catcode* regimen.

```
\cs_new_protected:Npn \__scontents_Scontents_norm_arg:n #1
  {
    \tl_set:Nn \l__scontents_Scontents_arg_tl {#1}
    \__scontents_Scontents_finish:
  }
```

(*End of definition for* `\__scontents_Scontents_norm_arg:n`.)

`\__scontents_Scontents_finish:` The function `\__scontents_Scontents_finish:` issues the "warning" `empty-argument-command` if the {⟨*argument*⟩} is empty, writes it with `\__scontents_file_write_cmd:VV` for the `write-out` and `write-cmd` keys and stores and prints it with `\__scontents_finish_storing:NNN`. Finally it closes the group and runs `\__scontents_esphack:` if the `print-cmd` key is not active.

```
\cs_new_protected:Nn \__scontents_Scontents_finish:
  {
    \tl_if_empty:NT \l__scontents_Scontents_arg_tl
      { \msg_warning:nn { scontents } { empty-argument-command } }
    \__scontents_file_write_cmd:VV \l__scontents_file_name_tl \l__scontents_Scontents_arg_tl
    \__scontents_finish_storing:NNN
      \l__scontents_Scontents_arg_tl \l__scontents_name_seq_cmd_tl \l__scontents_print_cmd_bool
    \use:e
      {
        \group_end:
        \bool_if:NF \l__scontents_print_cmd_bool { \__scontents_esphack: }
      }
  }
```

(*End of definition for* `\__scontents_Scontents_finish:`.)

`\Scontents` Defines the user command `\Scontents`.

```
\NewDocumentCommand \Scontents { !s !O{} }
  {
    \tl_set:Nn \l__scontents_cmd_name_tl { Scontents }
    \__scontents_Scontents_code:nn {#1} {#2}
  }
```

(*End of definition for* `\Scontents`. *This function is documented on page* 5.)

12.16 The command `\getstored`

The user-level command `\getstored` takes an optional argument with the ⟨*index*⟩, the last ⟨*stored content*⟩ by default, and a mandatory argument with the {⟨*seq name*⟩}. It is robust.

12.16.1 Internal functions for `\getstored`

`\__scontents_getstored:nn` The function `\__scontents_getstored:nn` rescans with `\tl_retokenize:e` the ⟨*stored content*⟩ at the ⟨*index*⟩ passed in `#1` in the *sequence* named by `#2`, given by `\__scontents_getfrom_seq:nn` (§12.11).

```
\cs_new_protected:Npn \__scontents_getstored:nn #1 #2
  {
    \tl_retokenize:e
      {
        \__scontents_getfrom_seq:nn {#1} {#2}
      }
  }
```

(*End of definition for* `\__scontents_getstored:nn`.)

`\getstored` Defines the user command `\getstored`.

```
\NewDocumentCommand \getstored { O{-1} m }
  {
    \__scontents_getstored:nn {#1} {#2}
  }
```

(*End of definition for* `\getstored`. *This function is documented on page* 6.)

©2019–2026 by Pablo González L

12.17 The command \foreachsc

The user-level command \foreachsc runs \getstored on the ⟨*stored contents*⟩ of a *sequence*. It takes an optional argument with the ⟨*keys*⟩ and a mandatory argument with the {⟨*seq name*⟩}, raising the "error" undefined-storage if the *sequence* does not exist.

12.17.1 Definition of variables for \foreachsc

\l__scontents_foreachsc_arg_tl
\l__scontents_foreach_print_seq
\g__scontents_foreach_exec_tl
\l__scontents_foreach_before_tl
\l__scontents_foreach_before_bool
\l__scontents_foreach_after_tl
\l__scontents_foreach_after_bool
\l__scontents_foreach_start_int
\l__scontents_foreach_step_int
\l__scontents_foreach_stop_int
\l__scontents_foreach_stop_bool
\l__scontents_foreach_wrapper_bool
\l__scontents_foreach_body_tl

The token list \l__scontents_foreachsc_arg_tl saves the {⟨*seq name*⟩}, the sequence \l__scontents_foreach_print_seq collects the code for each ⟨*stored content*⟩, built in the token list \l__scontents_foreach_body_tl, and the token list \g__scontents_foreach_exec_tl executes it outside the group. The other variables are set by the ⟨*keys*⟩ of \foreachsc.

```
\tl_new:N    \l__scontents_foreachsc_arg_tl
\seq_new:N   \l__scontents_foreach_print_seq
\tl_new:N    \g__scontents_foreach_exec_tl
\tl_new:N    \l__scontents_foreach_before_tl
\bool_new:N  \l__scontents_foreach_before_bool
\tl_new:N    \l__scontents_foreach_after_tl
\bool_new:N  \l__scontents_foreach_after_bool
\int_new:N   \l__scontents_foreach_start_int
\int_new:N   \l__scontents_foreach_step_int
\int_new:N   \l__scontents_foreach_stop_int
\bool_new:N  \l__scontents_foreach_stop_bool
\bool_new:N  \l__scontents_foreach_wrapper_bool
\tl_new:N    \l__scontents_foreach_body_tl
```

(*End of definition for* \l__scontents_foreachsc_arg_tl *and others.*)

12.17.2 Definition of keys for \foreachsc

before
after
start
stop
step
wrapper
sep
unknown

We define the ⟨*keys*⟩ for \foreachsc, the keys start, step and stop use __scontents_set_positive_int:Nnn (§12.8) and the unknown ⟨*keys*⟩ are passed to __scontents_unknown_keys_cmd:n (§12.9).

```
\keys_define:nn { scontents / foreachsc }
  {
    before  .code:n           = {
                                  \bool_set_true:N \l__scontents_foreach_before_bool
                                  \tl_set:Nn \l__scontents_foreach_before_tl {#1}
                                },
    before  .value_required:n = true,
    after   .code:n           = {
                                  \bool_set_true:N \l__scontents_foreach_after_bool
                                  \tl_set:Nn \l__scontents_foreach_after_tl {#1}
                                },
    after   .value_required:n = true,
    start   .code:n           = {
                                  \__scontents_set_positive_int:Nnn
                                    \l__scontents_foreach_start_int { start } {#1}
                                },
    start   .value_required:n = true,
    start   .initial:n        = 1,
    stop    .code:n           = {
                                  \__scontents_set_positive_int:Nnn
                                    \l__scontents_foreach_stop_int { stop } {#1}
                                  \bool_set:Nn \l__scontents_foreach_stop_bool
                                    {
                                      \int_compare_p:nNn
                                        { \l__scontents_foreach_stop_int } > { 0 }
                                    }
                                },
    stop    .value_required:n = true,
    step    .code:n           = {
                                  \__scontents_set_positive_int:Nnn
                                    \l__scontents_foreach_step_int { step } {#1}
                                },
    step    .value_required:n = true,
    step    .initial:n        = 1,
    wrapper .code:n           = {
                                  \bool_set_true:N \l__scontents_foreach_wrapper_bool
                                  \cs_set_protected:Npn
                                  \__scontents_foreach_wrapper:n ##1 {#1}
                                },
```

©2019–2026 by Pablo González L

```
    wrapper .value_required:n = true,
    sep     .tl_set:N         = \l__scontents_foreach_sep_tl,
    sep     .initial:n        = { },
    sep     .value_required:n = true,
    unknown .code:n           = { \__scontents_unknown_keys_cmd:n {#1} },
  }
```

(*End of definition for* `before` *and others.*)

12.17.3 Internal functions for `\foreachsc`

`\__scontents_foreachsc:nn`
`\__scontents_foreach_add_body:n`
`\__scontents_foreach_wrap:n`
`\__scontents_foreach_wrap:V`

The function `\__scontents_foreachsc:nn` sets the ⟨*keys*⟩ passed in `#1` and steps from `start` to `stop`, the number of ⟨*stored contents*⟩ by default, with `\__scontents_foreach_add_body:n`, which adds `\getstored`, with the code of the `before`, `after` and `wrapper` keys, to `\l__scontents_foreach_print_seq`. The result, separated by `sep`, is executed after the group. The function `\__scontents_foreach_wrap:n` expands `\__scontents_foreach_wrapper:n` once inside the group, so the code of the `wrapper` key is already in place when the result is executed; the function `\__scontents_foreach_wrapper:n` is local and no longer exists after the group.

```
\cs_new_protected:Npn \__scontents_foreachsc:nn #1 #2
  {
    \group_begin:
      \tl_if_novalue:nF {#1} { \keys_set:nn { scontents / foreachsc } {#1} }
      \tl_set:Nn \l__scontents_foreachsc_arg_tl {#2}
      \seq_clear:N \l__scontents_foreach_print_seq
      \bool_if:NF \l__scontents_foreach_stop_bool
        {
          \int_set:Nn \l__scontents_foreach_stop_int
            { \seq_count:c { g__scontents_name_#2_seq } }
        }
      \int_step_function:nnnN
        { \l__scontents_foreach_start_int }
        { \l__scontents_foreach_step_int }
        { \l__scontents_foreach_stop_int }
        \__scontents_foreach_add_body:n
      \tl_gset:Ne \g__scontents_foreach_exec_tl
        {
          \exp_args:NNV \seq_use:Nn
            \l__scontents_foreach_print_seq \l__scontents_foreach_sep_tl
        }
    \group_end:
    \exp_after:w \tl_gclear:N
    \exp_after:w \g__scontents_foreach_exec_tl
    \g__scontents_foreach_exec_tl
  }
\cs_new_protected:Npn \__scontents_foreach_add_body:n #1
  {
    \tl_set:Ne \l__scontents_foreach_body_tl
      { \exp_not:N \getstored [#1] { \exp_not:V \l__scontents_foreachsc_arg_tl } }
    \bool_if:NT \l__scontents_foreach_wrapper_bool
      { \__scontents_foreach_wrap:V \l__scontents_foreach_body_tl }
    \seq_put_right:Ne \l__scontents_foreach_print_seq
      {
        \bool_if:NT \l__scontents_foreach_before_bool
          { \exp_not:V \l__scontents_foreach_before_tl }
        \exp_not:V \l__scontents_foreach_body_tl
        \bool_if:NT \l__scontents_foreach_after_bool
          { \exp_not:V \l__scontents_foreach_after_tl }
      }
  }
\cs_new_protected:Npn \__scontents_foreach_wrap:n #1
  { \tl_set:No \l__scontents_foreach_body_tl { \__scontents_foreach_wrapper:n {#1} } }
\cs_generate_variant:Nn \__scontents_foreach_wrap:n { V }
```

(*End of definition for* `\__scontents_foreachsc:nn`*,* `\__scontents_foreach_add_body:n`*, and* `\__scontents_foreach_wrap:n`*.*)

`\foreachsc`

Defines the user command `\foreachsc`.

```
\NewDocumentCommand \foreachsc { +o m }
  {
    \tl_set:Nn \l__scontents_cmd_name_tl { foreachsc }
    \seq_if_exist:cTF { g__scontents_name_#2_seq }
```

©2019–2026 by Pablo González L

```
      { \__scontents_foreachsc:nn {#1} {#2} }
      { \msg_error:nnn { scontents } { undefined-storage } {#2} }
  }
```

(*End of definition for* `\foreachsc`. *This function is documented on page 6.*)

12.18 The optional argument of `\typestored`, `\meaningsc` and `\mergesc`

The commands `\typestored` and `\meaningsc`, and `\mergesc` which uses them internally, accept a single *optional argument* with the ⟨*index*⟩, ⟨*1-end*⟩ or the range ⟨*start-stop*⟩ of the ⟨*stored contents*⟩ in the *sequence* along with the ⟨*keys*⟩, this avoids the awkward `\typestored[...][⟨keys⟩]{...}` syntax. The ⟨*indexes*⟩ and ranges are processed as unknown ⟨*keys*⟩ of `l3keys`[9].

12.18.1 Definition of variables for the optional argument

`\l__scontents_current_seq_name_str` `\l__scontents_index_str` `\l__scontents_range_start_tl` `\l__scontents_range_stop_tl`

The string variable `\l__scontents_current_seq_name_str` stores the name of the *sequence* passed to `\typestored` or `\meaningsc`, it is used by `\__scontents_parse_type_meaning_key:nn`. The string variable `\l__scontents_index_str` is used to check that an ⟨*index*⟩ only has digits and the token lists `\l__scontents_range_start_tl` and `\l__scontents_range_stop_tl` save the limits of a range ⟨*start-stop*⟩.

```
\str_new:N \l__scontents_current_seq_name_str
\str_new:N \l__scontents_index_str
\tl_new:N  \l__scontents_range_start_tl
\tl_new:N  \l__scontents_range_stop_tl
```

(*End of definition for* `\l__scontents_current_seq_name_str` *and others.*)

12.18.2 Definition of keys for the optional argument

`width-tab` `write-out` `overwrite` `print-cmd` `unknown`

We define the common ⟨*keys*⟩ for `\typestored` and `\meaningsc`. The `write-out`, `overwrite` and `print-cmd` keys are necessary for the implementation of `\mergesc` (§12.21) and the unknown ⟨*keys*⟩ are passed to `\__scontents_parse_type_meaning_key:n`.

```
\keys_define:nn { scontents / typemeaning }
  {
    width-tab .meta:nn           = { scontents } { width-tab = #1 },
    write-out .code:n            = {
                                     \bool_set_false:N \l__scontents_storing_bool
                                     \bool_set_true:N  \l__scontents_writing_bool
                                     \tl_set:Nn \l__scontents_file_name_tl {#1}
                                   },
    write-out .value_required:n = true,
    overwrite .meta:nn           = { scontents } { overwrite = #1 },
    overwrite .default:n         = true,
    print-cmd .bool_set:N        = \l__scontents_print_verb_style_bool,
    print-cmd .initial:n         = true,
    print-cmd .default:n         = true,
    unknown   .code:n            = { \__scontents_parse_type_meaning_key:n {#1} },
  }
```

(*End of definition for* `width-tab` *and others.*)

12.18.3 Internal functions for the optional argument

`\__scontents_parse_type_meaning_key:n` `\__scontents_parse_type_meaning_key:nn` `\__scontents_seq_count:n` `\__scontents_range_parser:nnn` `\__scontents_range_parser:nnnw` `\__scontents_range_limit:nn` `\__scontents_range_step:nn` `\__scontents_if_digits:nTF`

The function `\__scontents_parse_type_meaning_key:n` checks that the unknown ⟨*key*⟩ has no value, otherwise something odd as `1=1` was used and the "error" `cmd-key-value-unknown` is raised. The function `\__scontents_seq_count:n` returns the number of ⟨*stored contents*⟩ in the *sequence* named by `#1`, or `0` if it does not exist. The conditional `\__scontents_if_digits:nTF` tests if `#1` is not blank and only has digits, it removes the digits from `\l__scontents_index_str`, so a text that is not a number never reaches an *integer expression* and only the "error" of the package is raised.

The function `\__scontents_range_parser:nnn` receives the ⟨*index*⟩ or the range in `#1`, the number of ⟨*stored contents*⟩ in `#2` and the code that raises the "error" in `#3`, and splits `#1` at the hyphens with `\__scontents_range_parser:nnnw`: without hyphen it is an ⟨*index*⟩, with a hyphen at the beginning it is a negative ⟨*index*⟩ and otherwise it is a range ⟨*start-stop*⟩, where `\__scontents_range_limit:nn` replaces `end` by the number of ⟨*stored contents*⟩ and `\__scontents_range_step:nn` adds the ⟨*indexes*⟩ of the range, ascending or descending, to `\l__scontents_seq_item_seq`. Anything else is passed to the code in `#3`.

```
\cs_new_protected:Npn \__scontents_parse_type_meaning_key:n #1
  { \exp_args:NV \__scontents_parse_type_meaning_key:nn \l_keys_key_str {#1} }
\cs_new_protected:Npn \__scontents_parse_type_meaning_key:nn #1#2
  {
```

```
    \tl_if_blank:nTF {#2}
      {
        \__scontents_range_parser:nnn {#1}
          { \__scontents_seq_count:n { \l__scontents_current_seq_name_str } }
          { \msg_error:nnn { scontents } { cmd-key-unknown } }
      }
      { \msg_error:nnnn { scontents } { cmd-key-value-unknown } {#1} {#2} }
  }
\cs_new:Npn \__scontents_seq_count:n #1
  {
    \seq_if_exist:cTF { g__scontents_name_#1_seq }
      { \seq_count:c { g__scontents_name_#1_seq } }
      { 0 }
  }
\prg_new_protected_conditional:Npnn \__scontents_if_digits:n #1 { TF }
  {
    \str_set:Ne \l__scontents_index_str { \tl_trim_spaces:n {#1} }
    \clist_map_inline:nn { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 }
      { \str_remove_all:Nn \l__scontents_index_str {##1} }
    \bool_lazy_or:nnTF
      { \tl_if_blank_p:n {#1} }
      { ! \str_if_empty_p:N \l__scontents_index_str }
      { \prg_return_false: }
      { \prg_return_true: }
  }
\prg_generate_conditional_variant:Nnn \__scontents_if_digits:n { V } { TF }
\cs_new_protected:Npn \__scontents_range_parser:nnn #1 #2 #3
  { \__scontents_range_parser:nnnw {#1} {#2} {#3} #1 - \q__scontents_mark - \s__scontents_mark }
\cs_new_protected:Npn \__scontents_range_parser:nnnw #1 #2 #3 #4 - #5 - #6 \s__scontents_mark
  {
    \__scontents_tl_if_head_is_q_mark:nTF {#5}
      {
        \__scontents_if_digits:nTF {#4}
          { \seq_put_right:Ne \l__scontents_seq_item_seq { \int_eval:n {#4} } }
          { #3 {#1} }
      }
      {
        \__scontents_tl_if_head_is_q_mark:nTF {#6}
          {
            \tl_if_blank:nTF {#4}
              {
                \__scontents_if_digits:nTF {#5}
                  {
                    \seq_put_right:Ne \l__scontents_seq_item_seq
                      { \int_eval:n { - #5 } }
                  }
                  { #3 {#1} }
              }
              {
                \tl_set:Ne \l__scontents_range_start_tl
                  { \__scontents_range_limit:nn {#4} {#2} }
                \tl_set:Ne \l__scontents_range_stop_tl
                  { \__scontents_range_limit:nn {#5} {#2} }
                \__scontents_if_digits:VTF \l__scontents_range_start_tl
                  {
                    \__scontents_if_digits:VTF \l__scontents_range_stop_tl
                      {
                        \exp_args:NVV \__scontents_range_step:nn
                          \l__scontents_range_start_tl \l__scontents_range_stop_tl
                      }
                      { #3 {#1} }
                  }
                  { #3 {#1} }
              }
          }
          { #3 {#1} }
      }
  }
\cs_new:Npn \__scontents_range_limit:nn #1 #2
  {
    \str_if_eq:eeTF { \tl_trim_spaces:n {#1} } { end }
```

```
      {#2}
      { \tl_trim_spaces:n {#1} }
  }
\cs_new_protected:Npn \__scontents_range_step:nn #1 #2
  {
    \int_step_inline:nnnn {#1}
      { \int_compare:nNnTF {#2} < {#1} { -1 } { 1 } }
      {#2}
      { \seq_put_right:Nn \l__scontents_seq_item_seq {##1} }
  }
```

(*End of definition for* `\__scontents_parse_type_meaning_key:n` *and others.*)

12.19 The command \typestored

The user-level command `\typestored` prints the ⟨*stored contents*⟩ of a *sequence* in the `verbatimsc` environment (§12.14). It takes an optional argument (§12.18) and a mandatory argument with the {⟨*seq name*⟩}.

12.19.1 Definition of variables for \typestored

`\l__scontents_typestored_arg_tl` The token list `\l__scontents_typestored_arg_tl` saves the ⟨*stored contents*⟩ to print.

```
\tl_new:N \l__scontents_typestored_arg_tl
```

(*End of definition for* `\l__scontents_typestored_arg_tl`.)

12.19.2 Definition of error messages for \typestored

`empty-variable` The "error" `empty-variable` is raised when the ⟨*stored contents*⟩ to print are blank.

```
\msg_new:nnnn { scontents } { empty-variable }
  { Blank~stored~content~to~print~\msg_line_context:. }
  {
    The~stored~content~only~has~spaces,~so~there~is~nothing~to~print~
    in~verbatim~style.~Check~the~stored~content~or~the~index.
  }
```

(*End of definition for* `empty-variable`.)

12.19.3 Internal functions for \typestored

`\__scontents_typestored:nn` The function `\__scontents_typestored:nn` sets the ⟨*keys*⟩ and the ⟨*indexes*⟩ passed in `#1`, the first ⟨*stored content*⟩ by default, gets the ⟨*stored contents*⟩ of the *sequence* named by `#2`, writes them to the `.log` file if `debug-var` is true and prints them with `\__scontents_typestored:N` and writes them with `\__scontents_file_write_cmd:VV`.

```
\cs_new_protected:Npn \__scontents_typestored:nn #1 #2
  {
    \__scontents_bsphack:
    \group_begin:
      \seq_clear:N \l__scontents_seq_item_seq
      \str_set:Ne \l__scontents_current_seq_name_str {#2}
      \tl_if_novalue:nF {#1}
        { \keys_set:nn { scontents / typemeaning } {#1} }
      \seq_if_empty:NT \l__scontents_seq_item_seq
        { \seq_set_from_clist:Nn \l__scontents_seq_item_seq { 1 } }
      \tl_set:Ne \l__scontents_typestored_arg_tl
        { \__scontents_getfrom_seq:Nn \l__scontents_seq_item_seq {#2} }
      \__scontents_remove_trailing_eol:N \l__scontents_typestored_arg_tl
      \tl_replace_all:NeV \l__scontents_typestored_arg_tl
        { \c__scontents_hidden_space_str } \c__scontents_newline_tl
      \bool_if:NT \l__scontents_debug_var_bool
        {
          \tl_log:N \l__scontents_typestored_arg_tl
        }
      \tl_if_empty:NF \l__scontents_typestored_arg_tl
        {
          \bool_if:NT \l__scontents_print_verb_style_bool
            {
              \__scontents_typestored:N \l__scontents_typestored_arg_tl
            }
        }
      \__scontents_file_write_cmd:VV \l__scontents_file_name_tl \l__scontents_typestored_arg_tl
```

```
      \use:e
        {
          \group_end:
          \bool_if:NF \l__scontents_print_verb_style_bool { \__scontents_esphack: }
        }
    }
```

(*End of definition for* `\__scontents_typestored:nn`.)

`\__scontents_typestored:N` `\__scontents_verb_print_EOL:`

The function `\__scontents_typestored:N` is defined with active carriage return characters to mimic an actual `verbatim` environment "on the loose". The content of the token list passed in `#1` is placed in a `verbatimsc` environment and rescanned with `\tl_retokenize:e`.

```
\group_begin:
  \char_set_catcode_active:N \^^M
  \cs_new_protected:Npn \__scontents_typestored:N #1
    {
      \tl_if_blank:VT #1
        { \msg_error:nn { scontents } { empty-variable } }
      \cs_set_eq:NN \__scontents_verb_print_EOL: ^^M
      \cs_set_eq:NN ^^M \scan_stop:
      \cs_set_eq:cN { do@noligs } \__scontents_do_noligs:N
      \int_set:Nn \tex_newlinechar:D { `\^^J }
      \tl_retokenize:e
        {
          \__scontents_format_case:nnn
            { \exp_not:N \begin{verbatimsc} } % LaTeX
            { \verbatimsc } % Plain/Generic
            { \startverbatimsc } % ConTeXt
            ^^M
          \exp_not:V #1 ^^M
          \g__scontents_end_verbatimsc_tl
        }
      \cs_set_eq:NN ^^M \__scontents_verb_print_EOL:
    }
\group_end:
```

(*End of definition for* `\__scontents_typestored:N` *and* `\__scontents_verb_print_EOL:`.)

`\typestored` Defines the user command `\typestored`.

```
\NewDocumentCommand \typestored { o m }
  {
    \tl_set:Nn \l__scontents_cmd_name_tl { typestored }
    \__scontents_typestored:nn {#1} {#2}
  }
```

(*End of definition for* `\typestored`. *This function is documented on page* 6.)

12.20 The command `\meaningsc`

The user-level command `\meaningsc` prints the meaning of the ⟨*stored contents*⟩ of a *sequence*. It takes the same arguments as `\typestored` (§12.19).

12.20.1 Definition of variables for `\meaningsc`

`\l__scontents_meaningsc_arg_tl` The token list `\l__scontents_meaningsc_arg_tl` saves the ⟨*stored contents*⟩ to print.

```
\tl_new:N \l__scontents_meaningsc_arg_tl
```

(*End of definition for* `\l__scontents_meaningsc_arg_tl`.)

12.20.2 Internal functions for `\meaningsc`

`\__scontents_meaningsc:nn` The function `\__scontents_meaningsc:nn` sets the ⟨*keys*⟩ and the ⟨*indexes*⟩ passed in `#1`, the first ⟨*stored content*⟩ by default, and calls `\__scontents_meaningsc:n` with the *sequence* named by `#2`.

```
\cs_new_protected:Npn \__scontents_meaningsc:nn #1 #2
  {
    \__scontents_bsphack:
    \group_begin:
      \seq_clear:N \l__scontents_seq_item_seq
      \str_set:Ne \l__scontents_current_seq_name_str {#2}
      \tl_if_novalue:nF {#1}
```

©2019–2026 by Pablo González L

```
      { \keys_set:nn { scontents / typemeaning } {#1} }
    \seq_if_empty:NT \l__scontents_seq_item_seq
      { \seq_set_from_clist:Nn \l__scontents_seq_item_seq { 1 } }
    \__scontents_meaningsc:n {#2}
    \use:e
      {
        \group_end:
        \bool_if:NF \l__scontents_print_verb_style_bool { \__scontents_esphack: }
      }
  }
```

(*End of definition for* `\__scontents_meaningsc:nn`.)

`\__scontents_meaningsc:n` The function `\__scontents_meaningsc:n` is defined with the `TAB` character active. It gets the ⟨*stored contents*⟩, replaces the ends of line and `\c__scontents_hidden_space_str` by spaces and each `TAB` character by as many spaces as the `width-tab` key was set to, prints them with `\cs_replacement_spec:N` and writes them with `\__scontents_file_write_cmd:VV`.

```
\group_begin:
  \char_set_catcode_active:N \^^I
  \cs_new_protected:Npn \__scontents_meaningsc:n #1
    {
      \tl_set:Ne \l__scontents_meaningsc_arg_tl
        { \__scontents_getfrom_seq:Nn \l__scontents_seq_item_seq {#1} }
      \tl_replace_all:Nen \l__scontents_meaningsc_arg_tl
        { \iow_char:N \^^J } { ~ }
      \tl_replace_all:Nen \l__scontents_meaningsc_arg_tl
        { \c__scontents_hidden_space_str } { ~ }
      \bool_if:NT \l__scontents_debug_var_bool
        {
          \tl_log:N \l__scontents_meaningsc_arg_tl
        }
      \tl_use:N \l__scontents_verb_font_tl
      \tl_replace_all:Nne \l__scontents_meaningsc_arg_tl
        { ^^I } { \prg_replicate:nn { \l__scontents_tab_width_int } { ~ } }
      \tl_if_empty:NF \l__scontents_meaningsc_arg_tl
        {
          \bool_if:NT \l__scontents_print_verb_style_bool
            {
              \cs_replacement_spec:N \l__scontents_meaningsc_arg_tl
            }
        }
      \__scontents_file_write_cmd:VV \l__scontents_file_name_tl \l__scontents_meaningsc_arg_tl
    }
\group_end:
```

(*End of definition for* `\__scontents_meaningsc:n`.)

`\meaningsc` Defines the user command `\meaningsc`.

```
\NewDocumentCommand \meaningsc { o m }
  {
    \tl_set:Nn \l__scontents_cmd_name_tl { meaningsc }
    \__scontents_meaningsc:nn {#1} {#2}
  }
```

(*End of definition for* `\meaningsc`. *This function is documented on page* 7.)

12.21 The command `\mergesc`

The user-level command `\mergesc` takes an optional argument with the ⟨*keys*⟩ and a mandatory argument with a *comma separated list* of {⟨*seq name*⟩} followed by an optional ⟨*index*⟩ or range. It assembles the ⟨*stored contents*⟩ in a temporary *internal sequence* and passes it to `\typestored` or `\meaningsc`.

12.21.1 Definition of variables for `\mergesc`

`\l__scontents_mergesc_arg_tl` `\l__scontents_mergesc_keys_tl` The token list `\l__scontents_mergesc_arg_tl` saves the last ⟨*stored content*⟩ assembled and the token list `\l__scontents_mergesc_keys_tl` saves the ⟨*keys*⟩ passed to `\typestored` or `\meaningsc`.

```
\tl_new:N \l__scontents_mergesc_arg_tl
\tl_new:N \l__scontents_mergesc_keys_tl
```

(*End of definition for* `\l__scontents_mergesc_arg_tl` *and* `\l__scontents_mergesc_keys_tl`.)

©2019–2026 by Pablo González L

12.21.2 Definition of error messages for \mergesc

mergesc-missing-key

The "error" mergesc-missing-key is raised when neither the typestored key nor the meaningsc key is passed to \mergesc.

```
\msg_new:nnnn { scontents } { mergesc-missing-key }
  {
    Missing~mandatory~key~'typestored'~or~'meaningsc'~for~
    \c_backslash_str mergesc~\msg_line_context:.
  }
  {
    The~command~\c_backslash_str mergesc~needs~one~of~the~keys~
    'typestored'~or~'meaningsc'.\\
    Check~that~you~have~spelled~the~key~name~correctly.
  }
```

(*End of definition for* mergesc-missing-key.)

12.21.3 Definition of keys for \mergesc

typestored
meaningsc
__scontents_mergesc_cmd:nn

The ⟨*keys*⟩ typestored and meaningsc are *mandatory*, they set the function __scontents_mergesc_cmd:nn, which by default raises the "error" mergesc-missing-key.

```
\keys_define:nn { scontents / mergesc }
  {
    typestored .code:n            =
      {
        \cs_set_eq:NN \__scontents_mergesc_cmd:nn \__scontents_typestored:nn
      },
    typestored .value_forbidden:n = true,
    meaningsc  .code:n            =
      {
        \cs_set_eq:NN \__scontents_mergesc_cmd:nn \__scontents_meaningsc:nn
      },
    meaningsc  .value_forbidden:n = true,
  }
\cs_new_protected:Npn \__scontents_mergesc_cmd:nn #1 #2
  {
    \msg_error:nn { scontents } { mergesc-missing-key }
  }
```

(*End of definition for* typestored, meaningsc, *and* __scontents_mergesc_cmd:nn.)

12.21.4 Internal functions for \mergesc

__scontents_mergesc_code:nn

The function __scontents_mergesc_code:nn sets the known ⟨*keys*⟩ passed in #1, saves the others in \l__scontents_mergesc_keys_tl, assembles the *internal sequence* from the list passed in #2 and, if it is not empty, passes it to __scontents_mergesc_cmd:nn.

```
\cs_new_protected:Npn \__scontents_mergesc_code:nn #1 #2
  {
    \group_begin:
      \tl_clear:N \l__scontents_mergesc_keys_tl
      \tl_if_novalue:nF {#1}
        {
          \keys_set_known:nnN { scontents / mergesc } {#1}
            \l__scontents_mergesc_keys_tl
        }
      \seq_gclear:c { g__scontents_name_sc!internal_seq }
      \__scontents_mergesc_parse_list:n {#2}
      \seq_if_empty:cF { g__scontents_name_sc!internal_seq }
        {
          \exp_args:Ne \__scontents_mergesc_cmd:nn
            { 1-end, \exp_not:V \l__scontents_mergesc_keys_tl } { sc!internal }
        }
    \group_end:
  }
```

(*End of definition for* __scontents_mergesc_code:nn.)

__scontents_mergesc_parse_list:n
__scontents_mergesc_parse_item:n
__scontents_parse_mergesc:nw
__scontents_parse_mergesc_aux:nw
__scontents_mergesc_put_item:nn

The function __scontents_mergesc_parse_list:n parses each item of the list with __scontents_mergesc_parse_item:n, which passes the {⟨*seq name*⟩} between braces to __scontents_parse_mergesc:nw when the item has no ⟨*index*⟩, because the *comma separated list* removes the braces around an item. The function __scontents_parse_mergesc:nw raises

the "error" `undefined-storage` if the *sequence* does not exist and uses all the ⟨*stored contents*⟩ of the *sequence* if there is no ⟨*index*⟩. The function `\__scontents_parse_mergesc_aux:nw` uses `\__scontents_range_parser:nnn` (§12.18) and `\__scontents_mergesc_put_item:nn` adds each ⟨*stored content*⟩ to the *internal sequence* or raises the "error" `index-out-of-range`. Finally the end of line of the last ⟨*stored content*⟩ is removed with `\__scontents_remove_trailing_eol:N`.

```
\cs_new_protected:Npn \__scontents_mergesc_parse_list:n #1
  {
    \clist_map_inline:nn {#1} { \__scontents_mergesc_parse_item:n {##1} }
    \seq_if_empty:cF { g__scontents_name_sc!internal_seq }
      {
        \seq_gpop_right:cN { g__scontents_name_sc!internal_seq } \l__scontents_mergesc_arg_tl
        \__scontents_remove_trailing_eol:N \l__scontents_mergesc_arg_tl
        \seq_gput_right:cV { g__scontents_name_sc!internal_seq } \l__scontents_mergesc_arg_tl
      }
  }
\cs_new_protected:Npn \__scontents_mergesc_parse_item:n #1
  {
    \tl_if_in:nnTF {#1} { [ } % ]
      { \__scontents_parse_mergesc:nw #1 \s__scontents_stop }
      { \__scontents_parse_mergesc:nw {#1} \s__scontents_stop }
  }
\cs_new_protected:Npn \__scontents_parse_mergesc:nw #1
  {
    \seq_if_exist:cTF { g__scontents_name_#1_seq }
      {
        \peek_remove_spaces:n
          {
            \peek_charcode:NTF [ % ]
              { \__scontents_parse_mergesc_aux:nw {#1} }
              { \__scontents_parse_mergesc_aux:nw {#1} [ 1-end ] }
          }
      }
      {
        \msg_error:nnn { scontents } { undefined-storage } {#1}
        \__scontents_use_none_delimit_by_s_stop:w
      }
  }
\cs_new_protected:Npn \__scontents_parse_mergesc_aux:nw #1 [#2]
  {
    \seq_clear:N \l__scontents_seq_item_seq
    \clist_map_inline:nn {#2}
      {
        \__scontents_range_parser:nnn {##1} { \__scontents_seq_count:n {#1} }
          { \msg_error:nnn { scontents } { cmd-key-unknown } }
      }
    \seq_map_inline:Nn \l__scontents_seq_item_seq
      { \__scontents_mergesc_put_item:nn {#1} {##1} }
    \__scontents_use_none_delimit_by_s_stop:w
  }
\cs_new_protected:Npn \__scontents_mergesc_put_item:nn #1 #2
  {
    \bool_lazy_or:nnTF
      { \int_compare_p:nNn {#2} = { 0 } }
      { \int_compare_p:nNn { \int_abs:n {#2} } > { \__scontents_seq_count:n {#1} } }
      {
        \msg_error:nneee { scontents } { index-out-of-range }
          {#2} {#1} { \__scontents_seq_count:n {#1} }
      }
      {
        \seq_gput_right:ce { g__scontents_name_sc!internal_seq }
          { \seq_item:cn { g__scontents_name_#1_seq } {#2} }
      }
  }
```

(*End of definition for* `\__scontents_mergesc_parse_list:n` *and others.*)

`\mergesc` Defines the user command `\mergesc`.

```
\NewDocumentCommand \mergesc { o m }
  {
    \tl_set:Nn \l__scontents_cmd_name_tl { mergesc }
```

©2019–2026 by Pablo González L

```
    \__scontents_mergesc_code:nn {#1} {#2}
  }
```

(*End of definition for* `\mergesc`. *This function is documented on page* 7.)

12.22 The command `\setupsc`

`\setupsc` Defines the user command `\setupsc`, which sets the ⟨*keys*⟩ of the package (§12.8).

```
\NewDocumentCommand \setupsc { +m }
  {
    \keys_set:nn { scontents } {#1}
  }
```

(*End of definition for* `\setupsc`. *This function is documented on page* 3.)

12.23 The command `\countsc`

`\countsc` Defines the user command `\countsc`, which counts the ⟨*stored contents*⟩ in the *sequence*. If the *sequence* does not exist, the "error" `undefined-storage` is raised and `0` is returned.

```
\NewExpandableDocumentCommand \countsc { m }
  {
    \seq_if_exist:cTF { g__scontents_name_#1_seq }
      { \seq_count:c { g__scontents_name_#1_seq } }
      {
        \msg_expandable_error:nnn { scontents } { undefined-storage } {#1}
        0
      }
  }
```

(*End of definition for* `\countsc`. *This function is documented on page* 8.)

12.24 The command `\cleanseqsc`

`\cleanseqsc` Defines the user command `\cleanseqsc`, which removes all the ⟨*stored contents*⟩ in the *sequence*.

```
\NewDocumentCommand \cleanseqsc { m }
  {
    \seq_gclear_new:c { g__scontents_name_#1_seq }
  }
⟨/core⟩
```

(*End of definition for* `\cleanseqsc`. *This function is documented on page* 8.)

12.25 Finish package

Finish package implementation.

```
⟨plain|context⟩\ExplSyntaxOff
⟨plain|context⟩\endinput
⟨latex|core⟩\file_input_stop:
```

©2019–2026 by Pablo González L

13 Index of Implementation

The italic numbers denote the pages where the corresponding entry is described, the numbers underlined and all others indicate the line on which they are implemented in the package code.

A

after 1115

B

before 1115

C

\cleanseqsc *8*, *53*, 1557
cmd-key-unknown 268
cmd-key-value-unknown 268
Commands provided by SCONTENTS:
\Scontents *43*
\cleanseqsc *53*
\countsc *53*
\foreachsc *45*
\getstored *43*
\meaningsc *50*
\mergesc *52*
\newenvsc *39*
\setupsc *53*
\typestored *49*
\countsc *8*, *53*, 1548

D

debug-var 208
\dospecials 904

E

empty-argument-command 1020
empty-environment 559
empty-store-name 182
empty-variable 1317
\endscontents *4*, 898
\endverbatimsc 968
env-already-defined 542
env-key-unknown 249
env-key-value-unknown 249
Environments provided by SCONTENTS:
verbatimsc *40*, *41*

F

force-eol 208, 565, 1022
\foreachsc *6*, *44*, 1204

G

\getstored *6*, *43*, 1098

I

index-out-of-range 164

J

junk-after-begin 542

M

\meaningsc *7*, *46*, *49*, 1429
meaningsc 1446
\mergesc *7*, *14*, *46*, *50*, 1539
mergesc-missing-key 1436
multiple-begin 559

N

\newenvsc *5*, *32*, 890
not-writing 191

O

overwrite 208, 565, 1022, 1215
overwrite-file 191

P

Packages :
expl3 . . . . . *21*
l3keys *46*
latex-lab *41*
ltcmd *20*, *33*, *42*
lua-widow-control *22*
verbatim *41*
xparse *20*
xsim *33*
xtemplate *41*
positive-integer 182
print-all 208
print-cmd 208, 1022, 1215
print-env 208, 565

Q

quark internal commands:
\q__scontents_mark 146
\q__scontents_stop 146

R

rescanning-text 559

S

scan internal commands:
\s__scontents_mark 146
\s__scontents_stop 146
\Scontents *5*, *41*, 1086
\scontents *4*, 898
scontents *4*, 898
scontents internal commands:
__scontents_analyse_nesting:n 763
__scontents_analyse_nesting_format:n . 785
__scontents_analyse_nesting_generic:nn 785
__scontents_analyse_nesting_generic:w 785
__scontents_analyse_nesting_generic_-process:nn 785
__scontents_analyse_nesting_latex:n . . 785
__scontents_analyse_nesting_latex:w . . 785
__scontents_append_contents:nn 454
__scontents_bsphack: 423
\l__scontents_char_value_int 12
__scontents_check_line_process:nn . . . 599
\l__scontents_cmd_name_tl 150
\l__scontents_current_seq_name_str . . . 1211
__scontents_define_env:nnn 858
__scontents_define_generic_nesting_-function:n 785
__scontents_do_noligs:N 382
\c__scontents_end_env_tl 118

©2019–2026 by Pablo González L

\g__scontents_end_verbatimsc_tl 118
__scontents_env_end_function: 638
\l__scontents_env_name_tl 118
\l__scontents_environment_keys_tl 536
__scontents_esphack: 423
__scontents_file_if_writable:nTF 338
\l__scontents_file_name_tl 151
__scontents_file_tl_write_start:n ... 638
__scontents_file_write_cmd:nn 361
\l__scontents_file_write_iow 151
__scontents_file_write_stop:N 638
__scontents_finish_storing:NNN 470
__scontents_foreach_add_body:n 1160
\l__scontents_foreach_after_bool 1102
\l__scontents_foreach_after_tl 1102
\l__scontents_foreach_before_bool 1102
\l__scontents_foreach_before_tl 1102
\l__scontents_foreach_body_tl 1102
\g__scontents_foreach_exec_tl 1102
\l__scontents_foreach_print_seq 1102
\l__scontents_foreach_start_int 1102
\l__scontents_foreach_step_int 1102
\l__scontents_foreach_stop_bool 1102
\l__scontents_foreach_stop_int 1102
__scontents_foreach_wrap:n 1160
\l__scontents_foreach_wrapper_bool ... 1102
__scontents_foreachsc:nn 1160
\l__scontents_foreachsc_arg_tl 1102
__scontents_format_case:nnn 97
__scontents_generic_begin: 844
__scontents_generic_end: 844
__scontents_getfrom_seq:Nn 484
__scontents_getfrom_seq:nn 484
__scontents_getfrom_seq:nNn 484
__scontents_getfrom_seq:nnn 484
__scontents_getfrom_seq_aux:nnn 484
__scontents_getstored:nn 1091
__scontents_grab_opt_arg:n 588
__scontents_grab_opt_arg:w 588
\c__scontents_hidden_space_str 160
__scontents_if_digits:nTF 1231
__scontents_if_nested:TF 763
\l__scontents_index_str 1211
\g__scontents_last_stored_tl 159
__scontents_lastfrom_seq:n 523
__scontents_make_control_chars_active: 393
__scontents_meaningsc:n 1402
__scontents_meaningsc:nn 1385
\l__scontents_meaningsc_arg_tl 1384
\l__scontents_mergesc_arg_tl 1434
__scontents_mergesc_cmd:nn 1446
__scontents_mergesc_code:nn 1463
\l__scontents_mergesc_keys_tl 1434
__scontents_mergesc_parse_item:n 1481
__scontents_mergesc_parse_list:n 1481
__scontents_mergesc_put_item:nn 1481
\g__scontents_name_sc!internal_seq ... 157
\l__scontents_nesting_aux_int 536
__scontents_nesting_decr: 763
\l__scontents_nesting_env_int 536
__scontents_nesting_incr: 763
\c__scontents_newline_tl 102
__scontents_nolig_list: 930
__scontents_normalise_line_ends:N ... 599
__scontents_par: 371
__scontents_parse_mergesc:nw 1481
__scontents_parse_mergesc_aux:nw 1481
__scontents_parse_type_meaning_key:n . 1231
__scontents_parse_type_meaning_key:nn 1231
\l__scontents_plain_bool 901
__scontents_plain_disable_outer_par: . 393
\l__scontents_processed_body_lines_tl . 536
__scontents_range_limit:nn 1231
__scontents_range_parser:nnn 1231
__scontents_range_parser:nnnw 1231
\l__scontents_range_start_tl 1211
__scontents_range_step:nn 1231
\l__scontents_range_stop_tl 1211
__scontents_remove_leading_nl:n 638
__scontents_remove_leading_nl:nn 638
__scontents_remove_leading_nl:w 638
__scontents_remove_trailing_eol:N ... 321
__scontents_remove_trailing_eol:w ... 321
\l__scontents_save_every_body_lines_tl 536
\l__scontents_save_sf_int 162
\l__scontents_save_skip 162
\l__scontents_Scontents_arg_tl 1018
__scontents_Scontents_code:nn 1045
__scontents_Scontents_finish: 1073
__scontents_Scontents_norm_arg:n 1068
__scontents_Scontents_verb_arg:w 1055
__scontents_seq_count:n 1231
\l__scontents_seq_item_seq 157
__scontents_set_active_eq:NN 393
__scontents_set_positive_int:Nnn 198
__scontents_setting_env:nn 858
__scontents_setup_verb_processor: ... 638
__scontents_start_after_option:w 599
__scontents_start_environment:w 599
__scontents_stop_environment: 599
__scontents_store_to_seq:NN 462
\l__scontents_storing_bool 151
__scontents_tab: 371
\l__scontents_tab_width_int 198
__scontents_tabs_to_spaces: 373
__scontents_tl_if_head_is_q_mark:nTF . 310
__scontents_typestored:N 1356
__scontents_typestored:nn 1323
\l__scontents_typestored_arg_tl 1316
__scontents_unknown_keys_cmd:n 298
__scontents_unknown_keys_cmd:nn 298
__scontents_unknown_keys_env:n 290
__scontents_unknown_keys_env:nn 290
__scontents_use_delimit_by_s_stop:nw . 306
__scontents_use_i_delimit_by_s_stop:nw 306
__scontents_use_none_delimit_by_q_-
stop:w 306
__scontents_use_none_delimit_by_s_-
stop:w 306
__scontents_verb_print_EOL: 1356
__scontents_verb_processor_iterate:nnn 638
__scontents_verb_processor_iterate:w . 638
__scontents_verb_processor_output:n .. 753
__scontents_verbatimsc_aux: 930

©2019–2026 by Pablo González L

__scontents_verbatimsc_instance: 980
\c__scontents_version_str 12
__scontents_vobeyspaces: 930
\l__scontents_writable_bool 151
\l__scontents_writing_bool 151
__scontents_xobeysp: 930
__scontents_xverb: 911
__scontents_xverb:w 911
sep 1115
\setupsc 3, 53, 1544
start 1115
\startscontents 4, 898
\startverbatimsc 977
step 1115
stop 1115
\stopscontents 4, 898
\stopverbatimsc 977
store-all 208
store-cmd 208, 1022
store-env 208, 565

T

tl commands:
\tl_if_empty:nTF 142
\tl_replace_all:Nnn 142
\tl_retokenize:n 142
\typestored 6, 46, 48, 1379
typestored 1446

U

undefined-storage 164
unknown 565, 1022, 1115, 1215

V

verb-font 208
\verbatimsc 968
verbatimsc 7, 995

W

width-tab 208, 1215
wrapper 1115
write-cmd 1022
write-env 565
write-out 565, 1022, 1215
writing-file 191

©2019–2026 by Pablo González L